1. DDU PROFILE

Dharmsinh DesaiUniversity

ISO 9001:2008

Nadiad - 387 001, Gujarat, India.

University (formerly known as DDIT), was established in June 1968 by Late Shri Dharmsinh Desai (then Member of Parliament), with a view to provide quality education in Engineering and Technology to the youth of Gujarat and was affiliated to Gujarat University.

Having modesty started with Diploma and Degree courses in Chemical Engineering. It has by now grown as top most Institute of Gujarat offering Graduate and Post Graduate level courses in various disciplines, and ultimately has become a State University in the year 2005. The institute gained an autonomous status in 1999 with approval from UGC and Gujarat Government and become a Deemed University accredited by N.B.A. in the year 2000.This University offers Graduate and Post Graduate level coursesin various disciplines and Master of Computer Applications (MCA) is being run under Faculty of Management and Information Science since 1987.

The University has also received ISO-9001:2008 Certification from ISOQAR and have also acquired NAAC accreditation. D.D.University believes not only in providing quality education, but also provides an overall development of its students in the form of being disciplined, sociality accountable to the society and responsible citizen of our county.

D.D. University is being guided by the vision laid down by its founding Chairman Late Shri Dharmsinh Desai, the present Chairman Dr. Narendra Desai and under a strong and dynamic leadership of Vice-chancellor Dr. H.M.Desai.

Vision:
"Become a multi-disciplined & „learner oriented" university to closely associate with and be responsive to the industry, to create supportive and caring environment for staff and students and to engage in R & D activities in areas of national priority."

Mission:
"Undertake programs and projects for development of human resources, both through formal and non-formal delivery systems, in areas of professional pursuits in all walks of human endeavors, with accent on relevance, value addition, societal needs and futuristic pilot projects."

2. COMPANY PROFILE

3. PROJECT PROFILE

Project Title	Online Shopping System
Company Name	
Tools	Java (Spring boot), React.js, Postman, IntelliJ IDEA, Vs Code
Duration	December 1, 2025 To January 18, 2025
Team Size	1
Team Member	Kartik Miteshbhai Patel
Internal Guide	Associate Professor Minal V. Shah
External Guide	-

4. TOOLS & TECHNOLOGY

Tools :-

Sr. No	Tools	Purpose
1	IntelliJ IDEA	Writing Code - Backend
2	Visual Studio Code	Writing Code – Frontend
3	MySQL	Database
4	Postman	API Testing
5	Junit, Mockito	Unit Testing
6		Deployment Server

Technology :-

Sr. No	Technology	Purpose
1	Spring Boot – Java	Backend Technology
2	React.js – JavaScript	Frontend Technology
3	MySQL command Line	Database

5. PROJECT PLANNING

5.1 Project Definition :

The **Online Shopping System (OSS)** is a full-stack, web-based e-commerce platform built using **Spring Boot (Java)** for the backend, **React.js** for the frontend, and **MySQL** as the relational database. The system is designed to provide customers with a seamless, 24/7 accessible online purchasing experience while empowering administrators with powerful tools for efficient store management. It digitizes and automates traditional retail operations such as product handling, inventory tracking, order processing, and sales reporting through a centralized, secure, and scalable platform.

The system eliminates the inefficiencies of manual product management, scattered inventory records, and time-consuming sales reporting by offering a structured digital solution. It brings together all core e-commerce operations under a single platform, ensuring transparency, accuracy, and ease of use for both administrators and customers.

The system supports **two primary user roles:**

Admin has full control over the system. The Admin can manage the complete product catalog by adding new products with details such as name, description, images, price, brand, category, and variants (size, colour, weight). The Admin can enable or disable products, manage categories and brands, monitor and update inventory stock levels, and receive low-stock alerts. The Admin can also view and manage all registered customer accounts, activate or deactivate accounts, view all placed orders, and generate business reports including sales reports, inventory reports, and product performance reports.

Customer can register on the platform using email and password or log in conveniently using **Google or GitHub OAuth2 social login**. Once logged in, the customer can browse the full product catalog, search for specific products by name, and apply filters by category, brand, and price range to narrow down results. The customer can view detailed product information including variants, descriptions, images, and reviews. They can add desired product variants to their shopping cart, update quantities, remove items, and view the complete cart summary with subtotals and grand total. Customers can also manage their personal profile including name, phone number, address, and city.

The system is built with **performance and security** as core priorities. **JWT-based authentication** ensures secure session management for standard login. **OAuth2 integration with Google and GitHub** simplifies the login process for users who prefer social login. **Spring Security** enforces role-based access control, ensuring that admin features are never accessible to customers and vice versa. All passwords are encrypted using **BCrypt** before being stored in the database. **Backend caching using Spring Cache (Caffeine)** is implemented to reduce database load and improve response times for frequently accessed data such as product listings and category information.

The **database design** follows a well-structured relational schema with key entities including User, Role, Product, ProductVariant, Brand, Category, Order, OrderItem, and ProductReview. **Spring Data JPA with Hibernate** is used for ORM-based database access, ensuring ACID-compliant transactional operations during cart and order management.

Example:

Consider a customer named Rahul who wishes to buy a gaming console. He visits the Online Shopping System and registers using his Google account via OAuth2 login no manual registration needed. He is redirected to the home page where he browses the "Consoles" category. He applies a filter

for the brand "Sony" and a price range of ₹40,000 to ₹60,000. The system instantly returns matching products from cache, displaying results without any delay.

Rahul selects the **PlayStation 5 : Spider-Man Edition** variant, reads the product description, checks the available stock, and adds 1 unit to his cart. The system verifies that the stock is available and updates the cart total to ₹50,000. Rahul reviews his cart, confirms the quantity, and proceeds.

Meanwhile, on the Admin side, the Admin logs into the dashboard and notices a **low-stock alert** for the PS5 Spider-Man Edition. The Admin updates the stock quantity and also adds a new product an Xbox Series X with full details, multiple variants, and images. The Admin also generates the monthly **Sales Report** as a PDF to review business performance.

This end-to-end flow demonstrates how the Online Shopping System brings efficiency, transparency, and a modern digital experience to both the store operator and the end customer.

5.2 Preliminary Investigation :

Before designing and developing the Online Shopping System, a preliminary investigation was carried out to understand the existing problems faced by both businesses and customers in traditional retail and manual e-commerce setups. This investigation helped identify the core pain points, justify the need for the system, and define the direction for the proposed solution.

Current System / Existing Problems :

- Traditional shopping requires customers to physically visit stores, which is time-consuming, geographically limited, and inconvenient especially for customers living far from commercial areas or those with busy schedules.
- Small and medium-sized businesses rely on manual methods such as registers, spreadsheets, or disconnected tools to manage their product catalog, stock levels, and sales records. This leads to frequent data errors, stock mismatches, and operational inefficiencies.
- There is no centralized platform where a business owner can simultaneously manage products, monitor inventory, view customer data, and analyze sales performance. Each task is handled separately, increasing effort and reducing accuracy.
- Inventory tracking done manually is highly error-prone. Businesses often face situations where products are shown as available but are actually out of stock, or vice versa, leading to customer dissatisfaction and lost sales.
- Product information such as price, description, images, and availability is scattered and inconsistent across different records. Updating product details requires manual effort across multiple places, increasing the risk of outdated or incorrect information being shown to customers.
- Customers have no way to search, filter, or compare products online. They must rely on physical catalogs, phone calls, or in-store visits to get product information, which is slow and inefficient.
- Sales reporting and business analysis are done manually at the end of the day or month, which is time-consuming and prone to human error. Business owners lack real-time insights into which products are selling well, which categories are performing poorly, and what the overall revenue looks like.

- There is no secure, role-based system in place. Anyone with access to the records can view or modify sensitive business data, creating security and privacy risks.
- Customer data such as contact information, purchase history, and account status is not maintained in a structured, centralized database. This makes customer relationship management difficult and inefficient.
- Businesses lack the tools to generate professional reports such as sales summaries, inventory status, or product performance analysis, which are essential for informed business decision-making.

Need for the Proposed System :

- There is a clear and urgent need for a centralized, web-based platform that brings together product management, inventory control, customer management, and sales reporting under one unified system.
- The system must be accessible 24 hours a day, 7 days a week, allowing customers to browse and shop at any time from any device without physical constraints.
- A secure authentication system using JWT and OAuth2 social login is required to protect user accounts and ensure that only authorized users can access system features.
- Role-based access control is essential to separate Admin and Customer functionalities, ensuring data integrity and system security.
- Automated inventory tracking is needed to eliminate manual stock management errors and provide real-time stock availability to customers.
- A structured product catalog with search and filtering capabilities is required to improve the customer shopping experience and help them quickly find the products they need.
- Automated report generation is necessary to give business owners instant access to sales data, inventory levels, and product performance metrics without manual effort.
- A responsive and user-friendly interface is required to make the system accessible to users on desktops, tablets, and mobile devices without any technical expertise.

Conclusion of Investigation :

The preliminary investigation clearly establishes that the existing manual and fragmented approaches to retail management are inadequate for the demands of modern business and customer expectations. The Online Shopping System directly addresses all identified problems by providing a structured, automated, secure, and scalable digital platform that benefits both the business (Admin) and the end user (Customer). The system is technically feasible, economically viable, and operationally practical, making its development fully justified.

6. SYSTEM ANALYSIS

6.1 Feasibility Study :

Before I started building the Online Shopping System, I took a step back and asked myself three important questions Can I actually build this with the technology I know? Will people find it useful and easy to use? And is it worth building from a cost perspective? The answers to these three questions form the feasibility study.

6.1.1 Technical Feasibility :

The first thing I checked was whether I have the right tools and technology to build this system on my own. The answer is yes and here is why.

For the backend, I chose **Spring Boot with Java**. It is one of the most popular and reliable frameworks used in real-world applications today. It handles things like API creation, security, database connection, and caching in a very clean and organized way. I did not have to build everything from scratch Spring Boot provides ready-made solutions for most backend needs, which saved me a lot of time and effort as a solo developer.

For the database, I used **MySQL**. It is a relational database which means data is stored in organized tables with proper relationships between them. For example, a product is linked to a category, a category is linked to a brand, and an order is linked to a customer. This structure makes sure that data stays consistent and accurate even when multiple things happen at the same time like two customers buying the same product simultaneously.

For the frontend, I used **React.js**. It allowed me to build a smooth and interactive user interface where pages update dynamically without reloading the entire browser window. Whether the user is browsing products, adding items to the cart, or updating their profile, everything feels fast and responsive. Since I was working alone, React's component-based structure helped me reuse UI components across different pages, saving significant development time.

For security, I implemented **JWT-based login** which means after a user logs in, they receive a secure token that is attached to every request they make. This ensures that only logged-in users can access protected pages. I also added **Google and GitHub login** using OAuth2 so users do not have to go through the hassle of creating a new account if they already have a Google or GitHub account.

I also added **backend caching** using Spring Cache. This means that data which is accessed very frequently like the product list or category list is stored temporarily in memory so the system does not have to hit the database every single time. This makes the system noticeably faster.

All of these technologies are open-source, freely available, and well-documented. I had the knowledge and the tools to work with all of them individually. Managing the full stack backend, frontend, and database on my own was challenging but completely achievable given the structured nature of these technologies and the clear separation between each layer of the system.

6.1.2 Operational Feasibility :

Technical feasibility tells us if I can build it. Operational feasibility tells us if people will actually use it and if it will genuinely make their lives easier. In this case, the answer is clearly yes.

Think about it from the **Admin's perspective**. Before this system, managing a store meant jumping between spreadsheets, manually counting stock, writing down orders, and spending hours preparing sales reports at the end of the month. With this system, everything is in one place. The admin logs in, sees the dashboard, and can immediately check how many products are available, which items are low on stock, how many customers are registered, and what orders have been placed. Adding a new product takes a few minutes fill in the details, upload an image, select the category and brand, set the price and stock, and it is live. Generating a sales report is just one click away. Everything that used to take hours now takes minutes.

Now think about it from the **Customer's perspective**. Instead of visiting a physical store or making phone calls to check if a product is available, the customer can simply open the website, search for what they want, filter by brand or price, and add it to their cart all from their phone or laptop at any time of the day. The login process is also very simple. Customers can use their existing Google or GitHub account to sign in without creating a new password. The website works well on mobile phones too, so customers are not limited to using it only on a desktop.

Since I designed and built this system myself, I paid close attention to making the interface as simple and intuitive as possible. Every screen, every button, and every error message was thought through from the user's point of view. The goal was to make sure that whether the person using it is a techsavvy developer or a shopkeeper with basic computer knowledge, they will find it easy to navigate without any training.

6.1.3 Economical Feasibility :

The last question is whether this system is worth building from a money perspective. And the answer is absolutely yes especially because almost everything I used to build it is free.

Every single technology I used Spring Boot, React.js, MySQL, Spring Security, Spring Cache, and all the related libraries is completely open-source. That means zero licensing fees. The development tools I used like IntelliJ IDEA Community Edition, Visual Studio Code, MySQL Workbench, and Postman are also free. So the entire cost of building this system came down to just my time and effort there was no financial investment required for software or tools.

Since I worked on this project alone, I had full control over every decision from the database schema design to the UI layout. This meant there was no miscommunication between team members, no duplicated work, and no wasted effort. Every hour I put in directly contributed to the final product.

When it comes to running the system, it can be hosted on free or very low-cost cloud platforms. The hardware required to run it is nothing special a basic server or even a mid-range laptop can handle it comfortably for a small to medium sized store.

Now let us talk about what a business actually saves by using this system. Inventory errors cost businesses money. If stock is incorrectly tracked, products get oversold or customers are turned away for items that are actually in stock. This system fixes that by keeping inventory updated

in real time. Sales reports that used to take an employee hours to prepare are now generated instantly with a single click. Time saved is money saved.

For a small business owner, this system means they no longer need to hire extra staff just to manage records, track stock, or prepare reports. Everything is automated. And since I built the codebase in a modular and well-organized way, adding new features in the future like a payment gateway, a discount system, or an order tracking feature does not require rebuilding the entire system from scratch. It can be done gradually without major additional investment.

In short, this system costs very little to build, very little to run, and saves a significant amount of time and money in day-to-day business operations. Building it single-handedly gave me complete ownership over the architecture and quality of every component, and the result is a system that is clean, efficient, and genuinely useful.

6.2 System Requirement Specification :

6.2.1 Introduction :

The Software Requirement Specification (SRS) for the Online Shopping System is a detailed document that describes what the system does, who uses it, what features it provides, and what conditions it must operate under. I prepared this document before starting the actual development so that I had a clear picture of everything I needed to build. It served as my blueprint throughout the entire project.

The **Online Shopping System (OSS)** is a web-based e-commerce platform that I designed and developed single-handedly using Spring Boot for the backend, React.js for the frontend, and MySQL as the database. The goal of this system is straightforward make online shopping easy for customers and make store management easy for administrators.

From the customer side, the system gives users the ability to browse a complete product catalog, search for specific items, filter results by category, brand, and price range, view detailed product information including variants and reviews, add products to a shopping cart, and manage their personal profile. Customers can register with their email or log in directly using their Google or GitHub account thanks to OAuth2 social login integration.

From the admin side, the system gives the administrator full control over the store. The admin can add and manage products with all their details and variants, organize products into categories, manage brands, monitor inventory levels, view and manage customer accounts, handle orders, and generate business reports such as sales reports, inventory reports, and product performance reports all from a single dashboard.

Before I started coding, I identified the following core modules that the system needed to have:

I. User Management Module : This module handles everything related to users registration, login, profile management, and role-based access. A user is either an Admin or a Customer, and the system behaves differently based on who is logged in. For example, a customer logging in sees the shopping interface while an admin logging in sees the management dashboard.

II. Product Management Module : This module allows the admin to manage the complete product catalog. Products can have multiple variants for example, a laptop can have variants based on RAM, storage, or color. Each variant has its own price, stock quantity, and image. The admin can add, update, enable, or disable products at any time.

III. Category Management Module : Products in the system are organized into categories. This module allows the admin to create, update, and delete categories. Categories help customers browse products more easily by grouping similar items together for example, Electronics, Clothing, or Gaming.

IV. Inventory Management Module : This module keeps track of how many units of each product variant are available in stock. When stock drops below a certain level, the system alerts the admin. Customers are also automatically prevented from adding out-of-stock products to their cart, which avoids confusion and overselling.

V. Cart Management Module : This module handles the shopping cart experience for customers. Customers can add products to their cart, change quantities, remove items, and see the running total of their cart at any time. The cart is tied to the customer's account so their selections are saved as long as they are logged in.

Each of these modules works independently but connects with the others seamlessly. For example, when a customer adds a product to their cart, the Cart module checks the Inventory module to confirm the stock is available before allowing the addition. This kind of cross-module interaction is what makes the system work as a complete, unified platform rather than a collection of disconnected features.

I built this system with the intention of solving real problems that small and medium businesses face when they try to sell products online. The system is not just a college project in terms of functionality it is built using industry-standard practices, follows a proper layered architecture, and covers all the core features that a functional e-commerce platform needs to have.

6.2.2 Purpose :

The purpose of the Online Shopping System is simple I wanted to build something that actually solves a real problem. The idea came from observing how small and medium businesses still rely on manual methods to manage their products, track their inventory, and record their sales. At the same time, customers are increasingly expecting a smooth, fast, and convenient online shopping experience. This system bridges that gap.

The primary purpose of this system is to provide a secure, centralized, and efficient platform where a business can run its entire online store and customers can shop without any friction.

From my perspective as the sole developer, I wanted to make sure that every feature I built had a clear reason behind it. I did not add things just to increase the feature count. Every module product management, inventory tracking, cart management, reporting exists because there is a genuine need for it in a real e-commerce scenario.

Here is what the system is specifically designed to do and why:

For the Admin:

- I built the product management module because businesses need a clean, structured way to add and maintain their product catalog. Without it, product information stays scattered across spreadsheets and sticky notes, which leads to errors and inconsistencies.
- I built the inventory management module because stock mismatches are one of the most common and costly problems in retail. A product showing as available when it is actually out of stock frustrates customers and damages the business's reputation. Real-time inventory tracking prevents this entirely.
- I built the category and brand management features because organized product data makes it easier for both the admin to manage listings and for customers to find what they are looking for. A well-structured catalog is the backbone of any good online store.
- I built the reporting module because business owners need data to make decisions. Without knowing which products are selling, which categories are performing well, and what the monthly revenue looks like, it is very difficult to run a store effectively. Automated report generation gives the admin this information instantly.
- I built the customer management module because admins need visibility over who is using their platform. Being able to view registered customers and activate or deactivate accounts gives the admin control over the user base.

For the Customer:

- I built the registration and login system with both JWT and OAuth2 because I wanted customers to have a smooth and secure entry into the platform. Nobody likes filling out long registration forms. With Google or GitHub login, a customer can be inside the store in seconds.
- I built the product browsing and filtering system because the biggest frustration in online shopping is not being able to find what you want quickly. With search, category filters, brand filters, and price range filters all working together, customers can narrow down thousands of products to exactly what they need in just a few clicks.
- I built the cart management system because the cart is the heart of the shopping experience. Customers need to be able to add items, adjust quantities, remove things they changed their mind about, and always see an accurate running total before they commit to a purchase.
- I built profile management because customers need to be able to keep their personal information up to date their address, phone number, and account details without having to contact anyone or go through a complicated process.

For Performance and Security:

- I implemented JWT-based authentication to make sure that every action a user takes on the platform is properly authorized. A customer should never be able to access admin pages and an admin should always have reliable, secure access to management features.
- I implemented backend caching because performance matters. A slow website loses customers. By caching frequently accessed data like product listings and categories, the system responds much faster and handles more users without putting excessive load on the database.
- I used BCrypt password encryption because storing plain text passwords is a serious security risk. Every password in the system is hashed before it is saved, so even if the database were ever compromised, the actual passwords would not be exposed.

In summary, the purpose of this system is to replace manual, error-prone, and fragmented retail management with a clean, automated, and unified digital platform and to give customers a shopping experience that is fast, convenient, and trustworthy.

6.2.3 Problem Statement :

Every project starts with a problem. The Online Shopping System started with several of them.

When I looked at how small and medium businesses manage their online or semi-online operations, I noticed a pattern of recurring problems that slow them down, cost them money, and frustrate their customers. These are the problems I set out to solve with this system.

Problem 1 : No centralized product management: Most small businesses manage their product information across multiple places a spreadsheet here, a WhatsApp message there, a handwritten note somewhere else. When a product's price changes or a new variant is added, updating all these places is time-consuming and mistakes are inevitable. There is no single source of truth for product data.

Problem 2 : Manual inventory tracking leads to errors: Tracking stock manually is unreliable. Businesses frequently end up in situations where a product is marked as available but is actually out of stock, or where stock runs out without anyone noticing until a customer tries to buy it. These errors lead to cancelled orders, unhappy customers, and lost revenue.

Problem 3 : Customers cannot shop conveniently: Without an online platform, customers have to physically visit the store or call to check product availability, pricing, and details. This is inconvenient, time-consuming, and limits the business to only serving customers who are physically nearby or willing to make the effort to call.

Problem 4 : No search or filtering capability: Even businesses that have some kind of online presence often lack proper search and filtering tools. Customers are forced to scroll through long lists of products trying to find what they want, which is a frustrating experience that often ends with them leaving the site empty-handed.

Problem 5 : Sales reporting is slow and inaccurate: Preparing a monthly sales report manually means going through every order, adding up totals, categorizing products, and compiling everything into a readable format. This process takes hours and is highly prone to human error. Business owners end up making decisions based on outdated or inaccurate data.

Problem 6 : No structured customer data: Without a proper system, customer information is stored inconsistently sometimes in a register, sometimes in a phone contact, sometimes nowhere at all. This makes it impossible to manage customer accounts, track purchase behavior, or take any kind of data-driven action.

Problem 7 : Security and access control are ignored: In manual systems, there is typically no concept of roles or permissions. Anyone with access to the records can view or change sensitive business data. There is no audit trail, no password protection, and no way to control who sees what.

Problem 8 : No performance optimization for growing data: As a business grows and the number of products, customers, and orders increases, manual systems completely break down. Even basic digital solutions without proper architecture struggle to handle growing data efficiently.

All of these problems point to the same root cause the absence of a proper, dedicated, and well-designed digital platform for managing an online store.

6.2.4 Scope :

The scope of the Online Shopping System defines exactly what the system covers, what it does not cover, and the boundaries within which it operates. Having a clear scope helped me stay focused during development and avoid scope creep the common trap of endlessly adding features without finishing the core ones.

What the system covers:

The system is a complete web-based e-commerce platform built to handle the core operations of an online store. It covers the following areas:

User Management: The system handles the complete user lifecycle from registration and login to profile management and account control. Customers can register with their email and password or sign in using their Google or GitHub account via OAuth2. Admins have a separate login and access to a fully different set of features. The system uses JWT tokens for session management, ensuring that every user's session is secure and properly authenticated.

Product Management: The system covers the complete product catalog management workflow. The admin can add new products with full details including name, description, category, brand, images, and status. Each product can have multiple variants with individual pricing, stock quantity, size, color, weight, and images. Products can be enabled or disabled at any time without being permanently deleted.

Category and Brand Management: The system covers the creation, updating, and deletion of product categories and brands. Categories and brands are linked to products, keeping the catalog well-organized and easy to navigate for customers.

Inventory Management: The system covers real-time stock tracking for every product variant. The admin can view stock levels, update quantities manually, and receive low-stock alerts. The system automatically prevents customers from adding out-of-stock items to their cart.

Cart Management: The system covers the complete shopping cart experience for customers. Customers can add products to their cart, update quantities, remove items, and view their cart total at any time. Cart data is linked to the customer's account and is maintained throughout their session.

Reporting: The system covers basic business reporting for the admin including sales reports, inventory reports, and product performance reports. These reports are generated automatically and can be exported as PDF documents.

Security: The system covers authentication, authorization, and data protection. JWT-based login, OAuth2 social login, BCrypt password encryption, and role-based access control are all within the scope of this system.

Performance: The system covers backend caching using Spring Cache with Caffeine for frequently accessed data. This ensures fast response times even under higher user loads.

What the system does not cover:

I made deliberate decisions about what to leave out of this version of the system. These features are not absent because they are unimportant they are planned as future enhancements once the core system is stable and complete.

- **Payment gateway integration :** Online payment processing via UPI, cards, wallets, or BNPL is not included in the current version. Orders can be placed but payment processing is out of scope for now.

- **Order tracking and delivery management :** Real-time tracking of orders after placement and integration with delivery services is not covered in this version.
- **Recommendation engine :** Personalized product recommendations based on browsing or purchase history are not part of the current scope.
- **Discount and coupon system :** Promotional pricing, coupon codes, and discount management are planned for a future release.
- **Mobile application :** The current system is a web application. A dedicated iOS or Android mobile app is outside the current scope.
- **Third-party logistics integration :** Connecting with courier or shipping APIs is not included in this version.

Why this scope makes sense:

As a solo developer working within a fixed timeline, defining a clear and realistic scope was one of the most important decisions I made. Rather than building a half-finished system with twenty features, I chose to build a complete, fully functional, and well-tested system covering the core e-commerce workflow. Every feature within the scope is properly implemented, tested, and connected to the rest of the system. The architecture is designed in a way that future features can be added cleanly without disrupting what already exists.

6.2.5 Audience and Users :

Every system is built for someone. Before I wrote a single line of code, I thought carefully about who would actually be using this system and what they would need from it. The Online Shopping System is designed for two types of users the Admin and the Customer. Each of them has a completely different relationship with the system, different goals, and different expectations.

I. Admin :

The Admin is the person who runs the store behind the scenes. In a real-world scenario, this would be the business owner, store manager, or a designated staff member responsible for managing the online platform.

The Admin does not shop the Admin manages. Everything the customer sees on the platform the products, the categories, the brands, the stock availability is controlled by the Admin. The Admin is also responsible for keeping the business data accurate and up to date. If a product goes out of stock, the Admin updates it. If a new product arrives, the Admin adds it. If a customer account needs to be deactivated, the Admin does it.

The Admin also has access to business intelligence features that customers never see. Reports on monthly sales, inventory levels, and product performance are all available exclusively to the Admin. These reports help the Admin understand how the business is doing and make informed decisions about what to stock, what to promote, and where to focus effort.

The Admin interacts with the system primarily through the Admin Dashboard a centralized management interface where all administrative features are accessible from one place. The Admin does not need to be a technical person. The dashboard is designed to be straightforward and practical, requiring no programming knowledge or special training.

II. Customer :

The Customer is the person the entire shopping experience is built for. In a real-world scenario, this could be anyone a student buying a laptop, a gamer looking for a console, or a professional shopping for accessories.

The Customer's interaction with the system is focused entirely on discovery and purchase. They come to the platform with a need, they search or browse to find the right product, they evaluate options by reading descriptions and checking variants, they add items to their cart, and they manage their profile. The experience is designed to be as smooth and frictionless as possible so that the customer can focus on finding what they want without getting confused or frustrated by the interface.

Customers access the system through the Customer-facing storefront the product browsing pages, the cart, the profile section, and the login and registration screens. Unlike the Admin, the Customer has no visibility into stock management, report generation, or any backend operations. What they see is a clean, organized online store.

One important thing I kept in mind while designing for customers is that not all customers are equally tech-savvy. Some might be very comfortable with technology while others might be using an online shopping platform for the first time. The interface accommodates both it is simple enough for a first-time user but efficient enough for someone who shops online regularly.

6.2.6 Overview :

6.2.6.1 Hardware and software (for development)

Hardware Requirements	
Processor	Intel core i5 or higher
RAM	8 GB
Software Requirements	
Backend	JDK, Intellij IDEA, MySQL, Node
Database	MySQL
Tools	V Vs Code, Intellij IDEA, MySQL, Postman
Browser	All Latest browser

6.2.6.2 Hardware and software (for User)

Hardware Requirements	
Processor	Intel core i3 or higher
RAM	4 GB
Software Requirements	
Operating System	Windows, Ios, Android
Browser	All Latest Browser

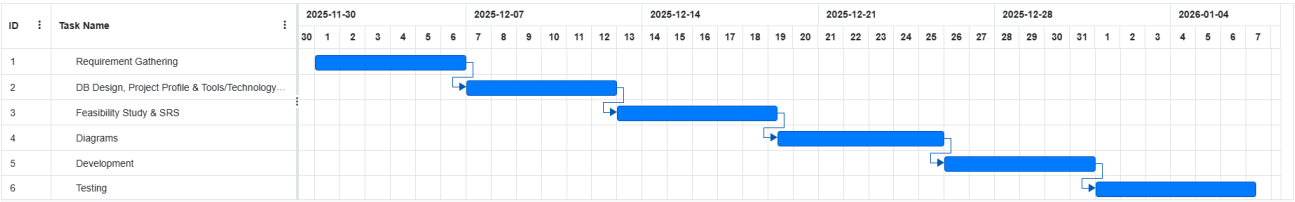
6.2.7 Acronyms and Abbreviations :

- IDE – Integrated Development Environment
- API – Application Programming Interface
- DB – Database
- JWT – JSON Web Token
- RBAC – Role-Based Access Control

6.2.8 Functional Requirements :

6.2.8.1 Other Information :

a. Gantt Chart :



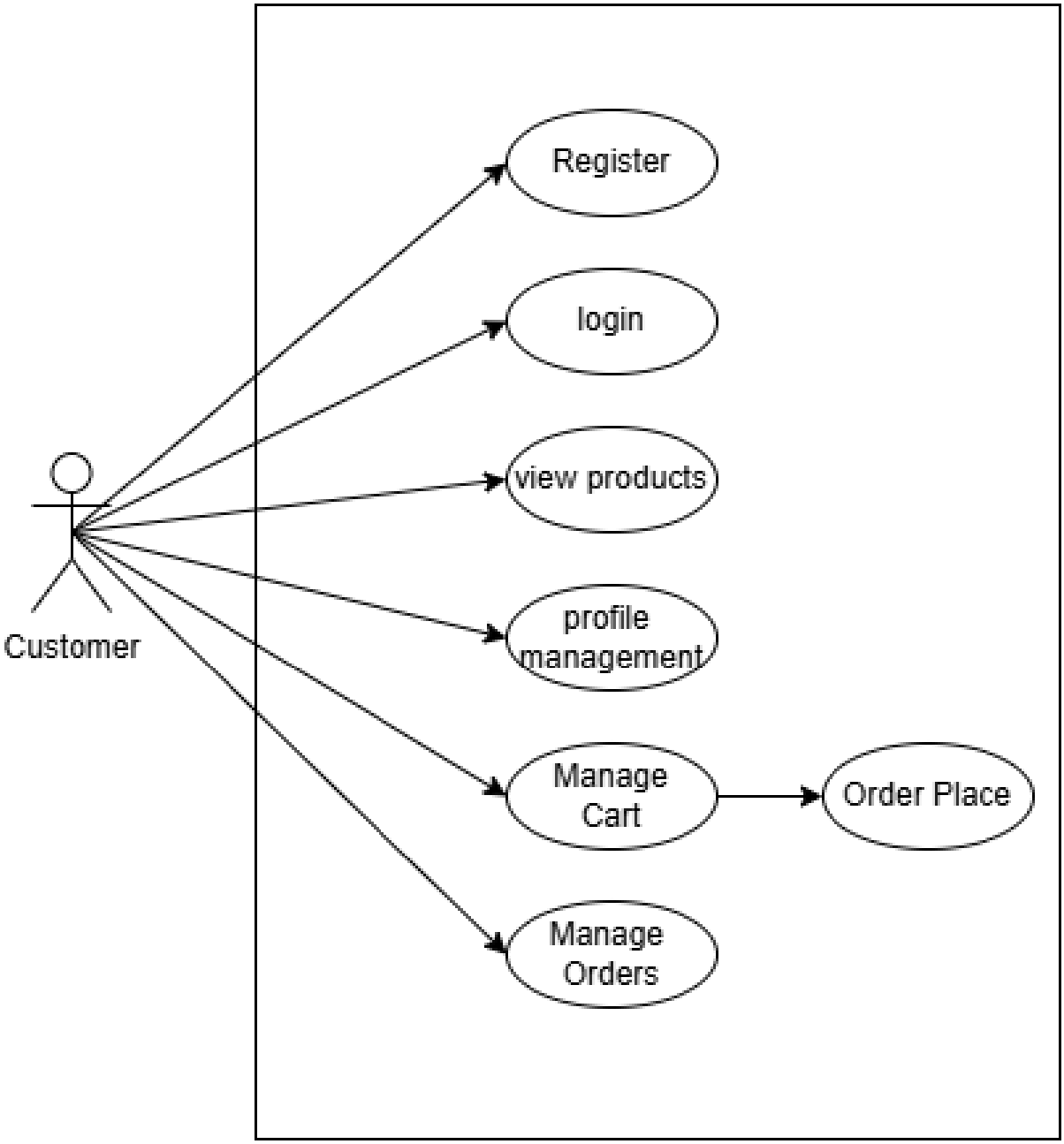
7 SYSTEM DESIGN

7.1 Architectural Design :

7.1.1 Use case Diagram :

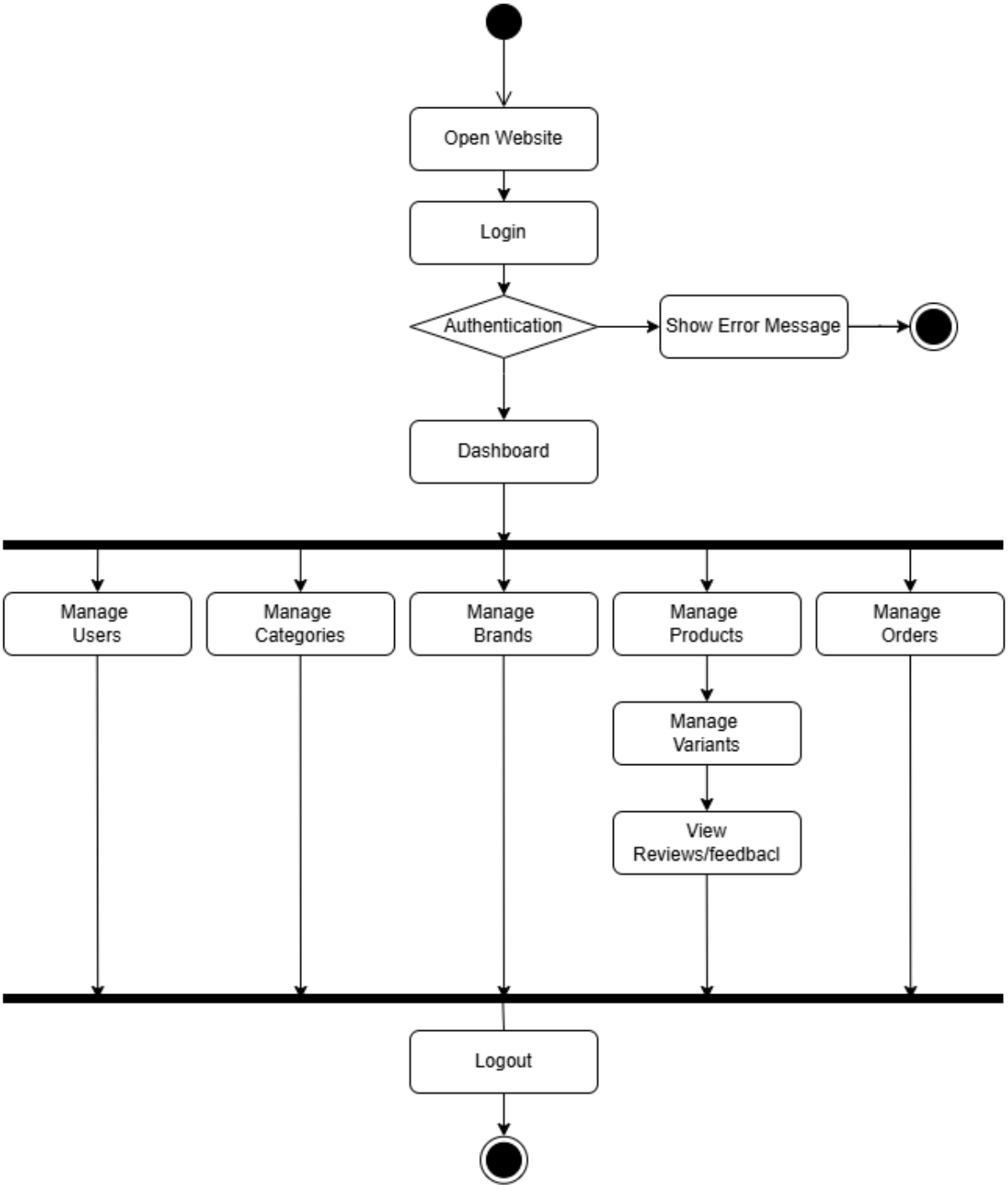
7.1.1.1 Admin Use case :

7.1.1.2 Customer Use case:

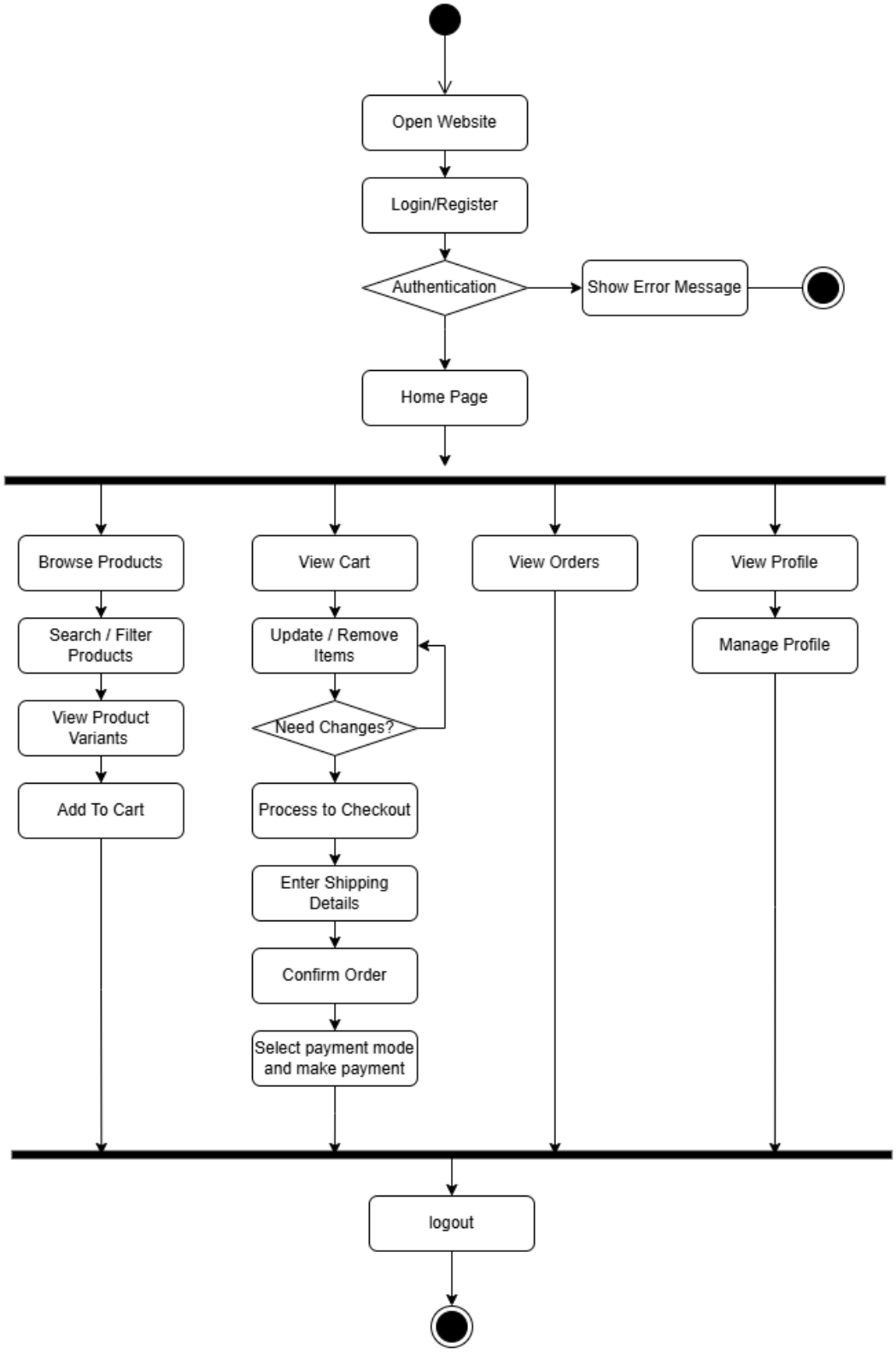


7.1.2 Activity Diagram :

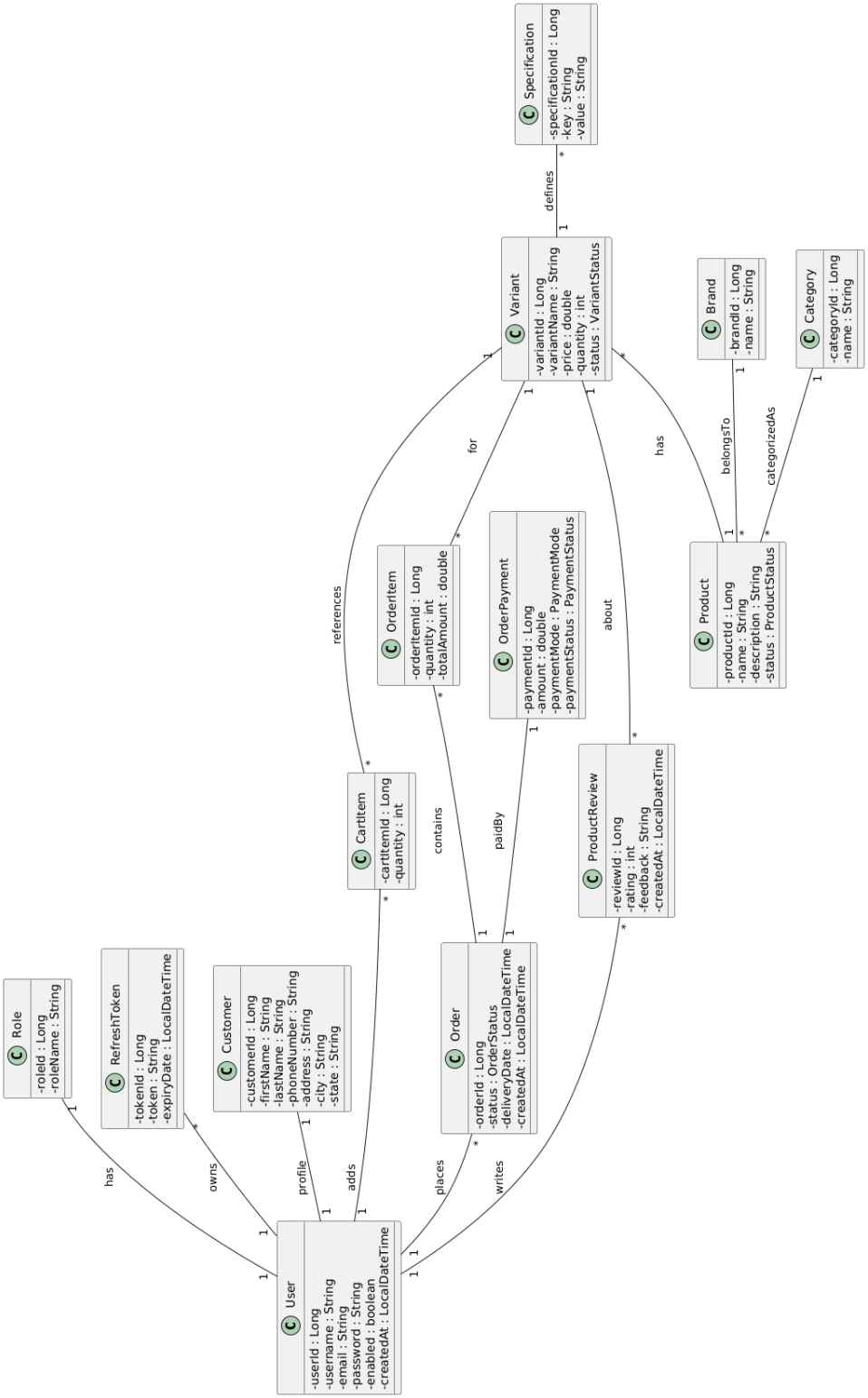
7.1.2.1 Admin Activity :



7.1.2.2 Customer Activity :

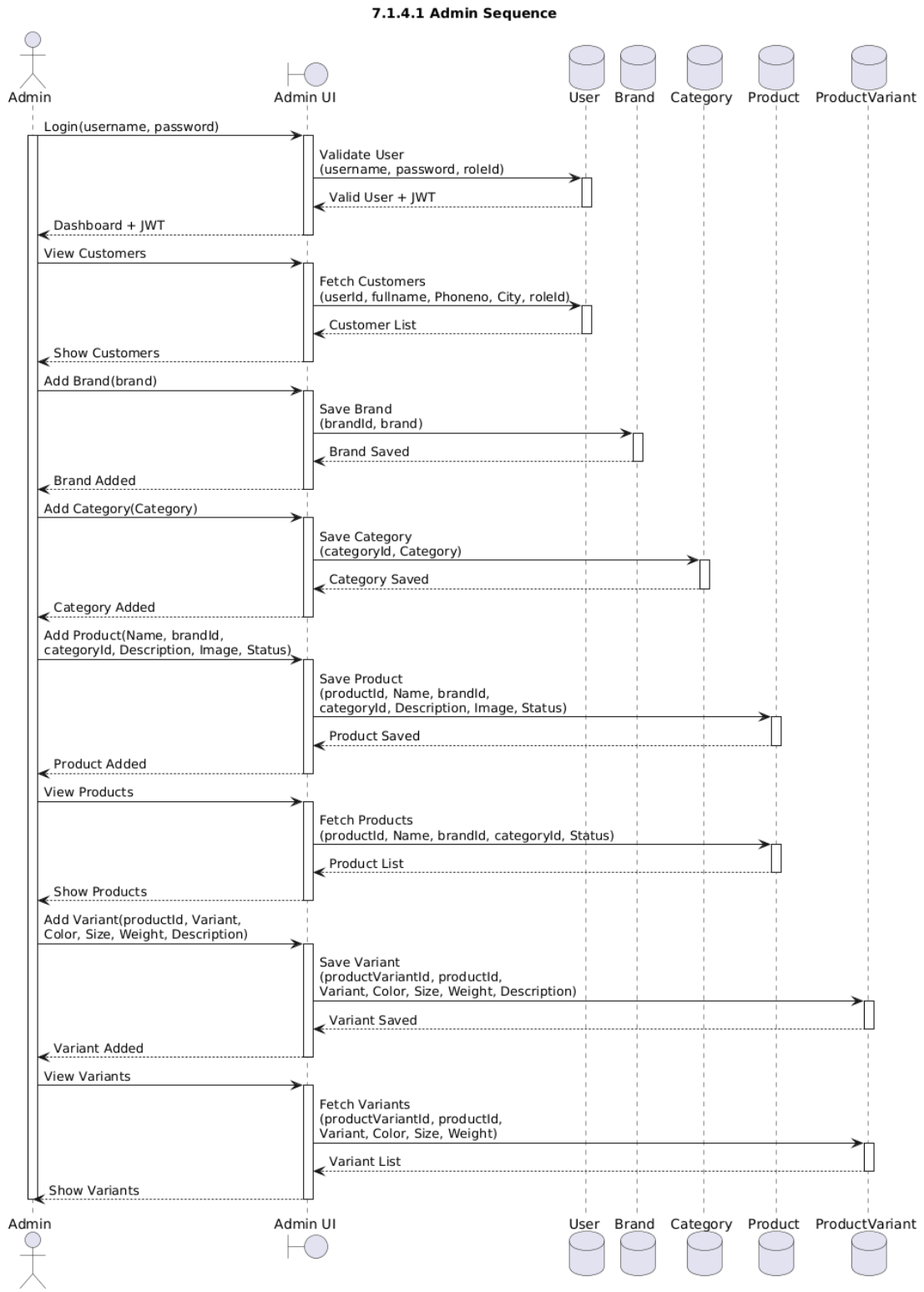


7.1.3 Class Diagram :

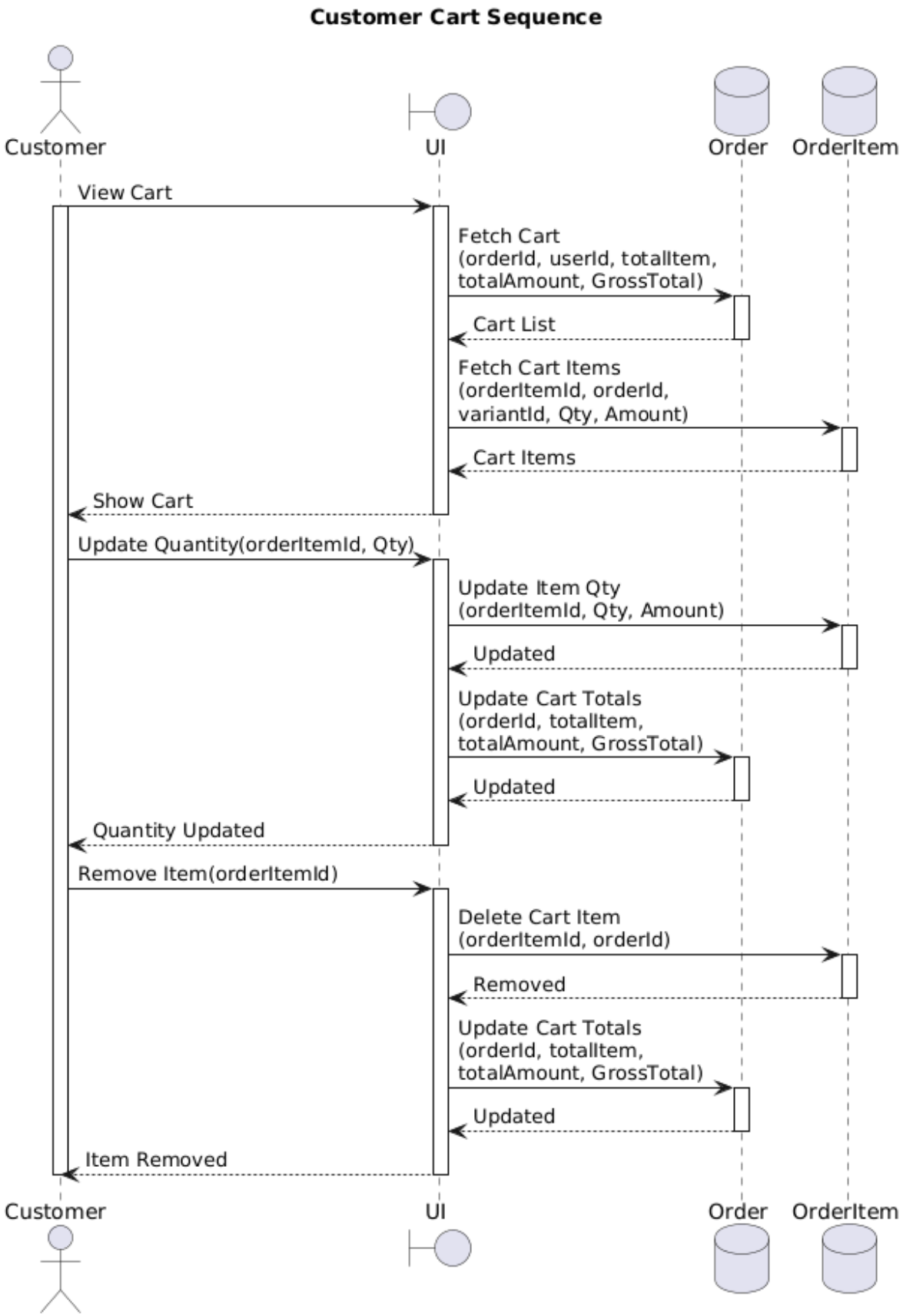


7.1.4 Sequence Diagram :

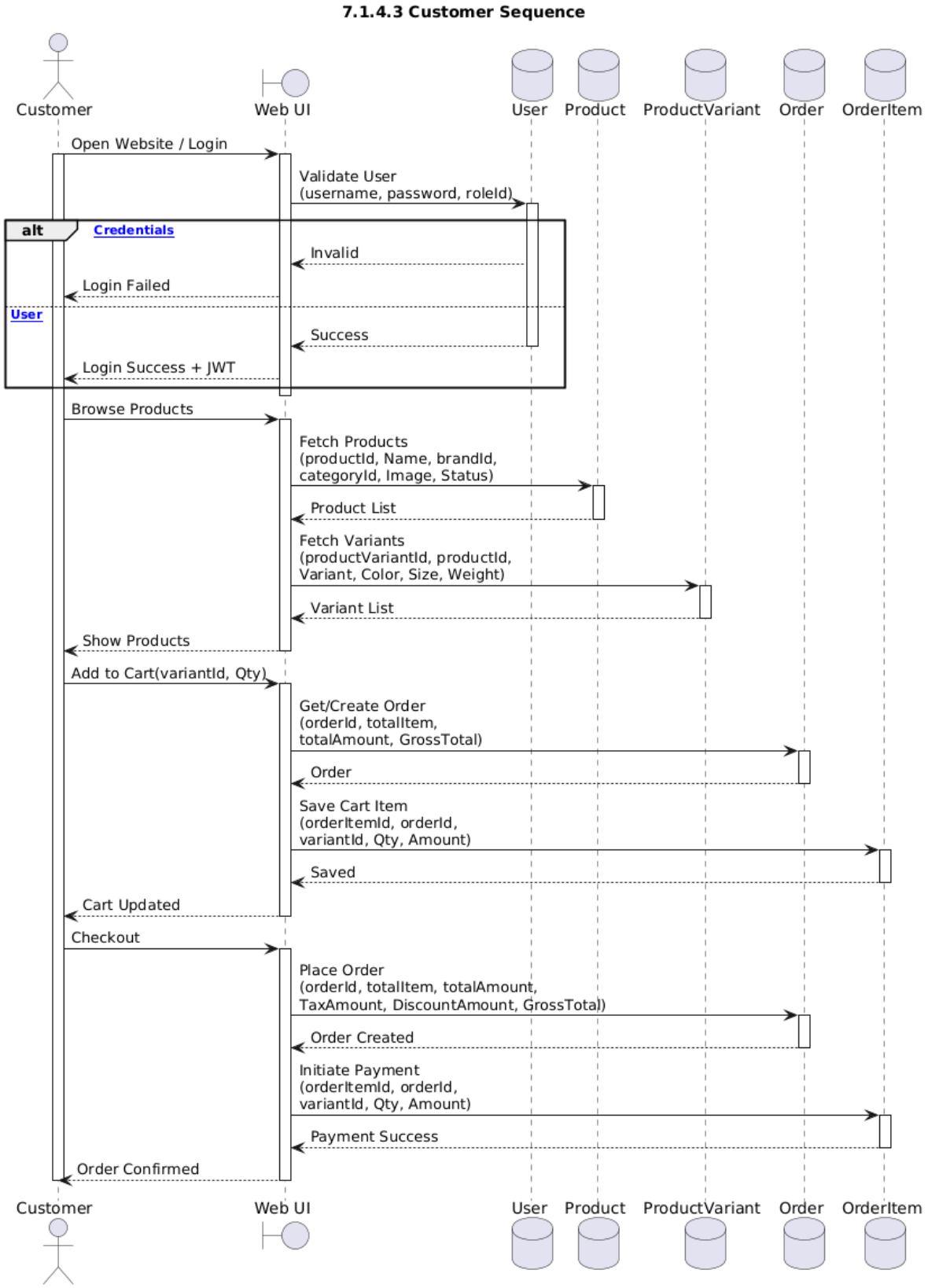
7.1.4.1 Admin Sequence :



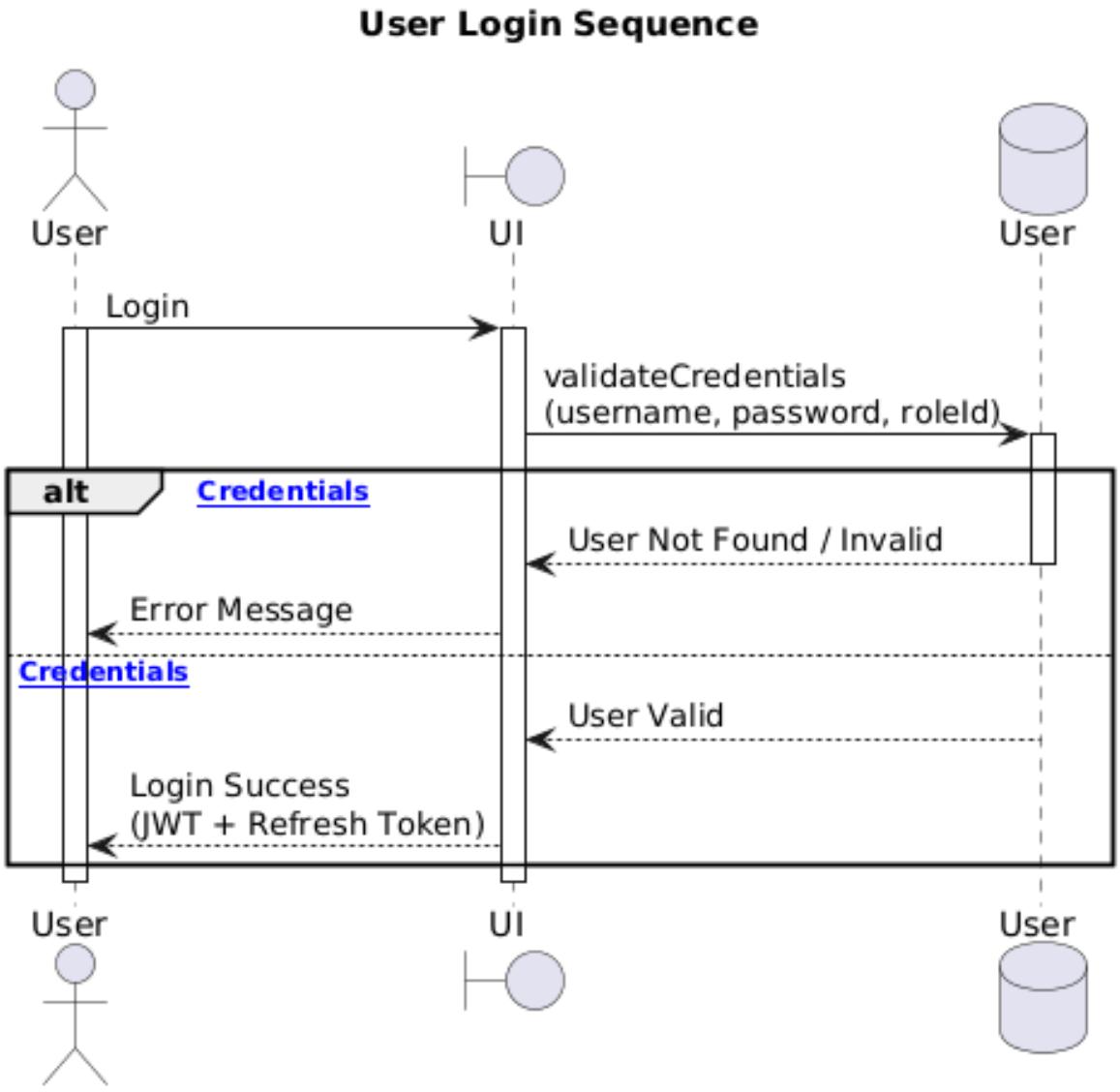
7.1.4.2 Cart Sequence :



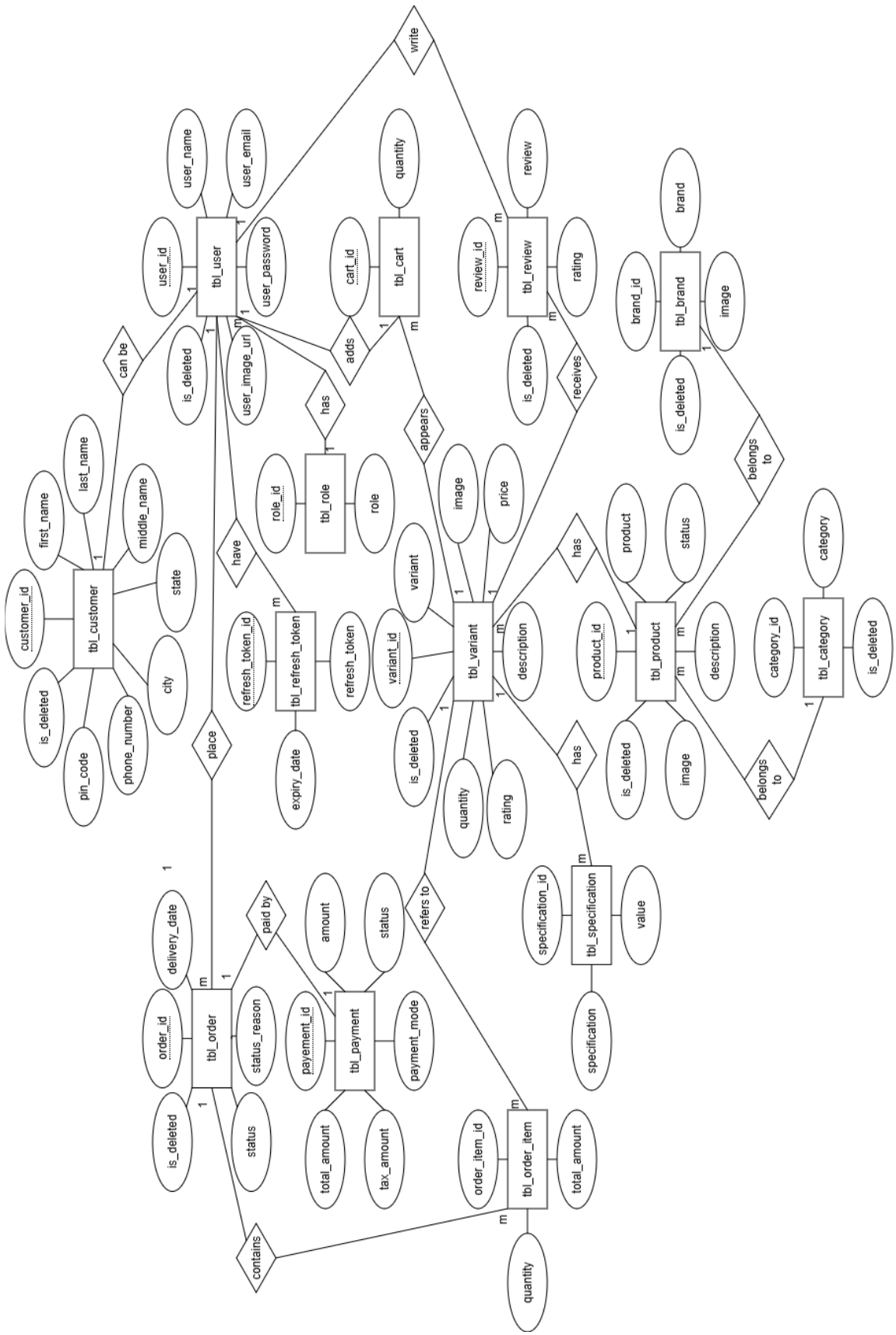
7.1.4.3 Customer Sequence :



7.1.4.4 Login Sequence :



7.2 ER Diagram :



7.3 Data Dictionary :

1. **Brand :** To Store brand Of Product

Sr. No	Field	Datatype	Constrain	Description
1	brandId	Number(4)	Primary Key	For Uniquely Identification
2	brand	Varchar(20)	Unique	Store Brand

2. **Category :** To store category of Product

Sr. No	Field	Datatype	Constrain	Description
1	categoryId	Number(4)	Primary Key	For Uniquely Identification
2	Category	Varchar(20)	Unique	Store Category

3. **Order :** To store Order Of Candidate

Sr. No	Field	Datatype	Constrain	Description
1	orderId	Number(4)	Primary key	For Uniquely Identification
2	totalItem	Number(2)		Store total Qty Of Items
3	totalAmount	Number(5,2)		Store total amount of order
4	TaxAmount	Number(5,2)		Tax Amount if any
5	DiscountAmount	Number(5,2)		Discount Amount if any
6	GrossTotal	Number(5,2)		Store overall Total Of Product

4. **Order Item :** To store Item Of Specific Order

Sr. No	Field	Datatype	Constrain	Description
1	orderItemId	Number(4)	Primary Key	For Uniquely Identification
2	orderId	Number(4)	Referenced(Order)	Store orderId
3	variantId	Number(4)	Referenced (ProductVariant)	Store productVariantId
4	Qty	Number(2)		Store Qty of the Order item
5	Amount	Number(5,2)		Store Total Amount of Items

5. **Product :** To store Product details

Sr. No	Field	Datatype	Constrain	Description
1	productId	Number(4)	Primary Key	For Uniquely Identification
2	Name	Varchar(30)		product Name
3	brandId	Number(4)	Referenced(Brand)	To store brandId
4	categoryId	Number(4)	Referenced(Category)	To store categoryId
5	Description	Text		About Product
6	Image	Text		Store image url of product
7	Status	Varchar(10)		Store status about product either Enabled/Disabled

6. **Product Review :** To store Review of specific Product

Sr. No	Field	Datatype	Constrain	Description
1	productReviewId	Number(4)	Primary Key	For Uniquely Identification
2	productVariantId	Number(4)	Referenced (ProductVariant)	Store productVarientId
3	Review	Text		Review of customer
4	Rating	Number(1)		Store start rating
5	reviewedBy	Number(4)	Referenced(User)	Store userId

7. **Product Variant :** To store Variant of Product

Sr. No	Field	Datatype	Constrain	Description
1	productVariantId	Number(4)	Primary Key	For Uniquely Identification
2	productId	Number(4)	Referenced(Product)	To Store ProductId
3	Variant	Varchar(50)		Store Variant Information
4	Color	Varchar(20)		Store Color Of Product Variant
5	Size	Varchar(10)		Store Size Of Product Variant
6	Weight	Number(3,2)		Store Weight Of Product Variant
7	Description	Text		Store detailed information about variant

8. Role : To store Role Of User Either ADMIN or CUSTOMER

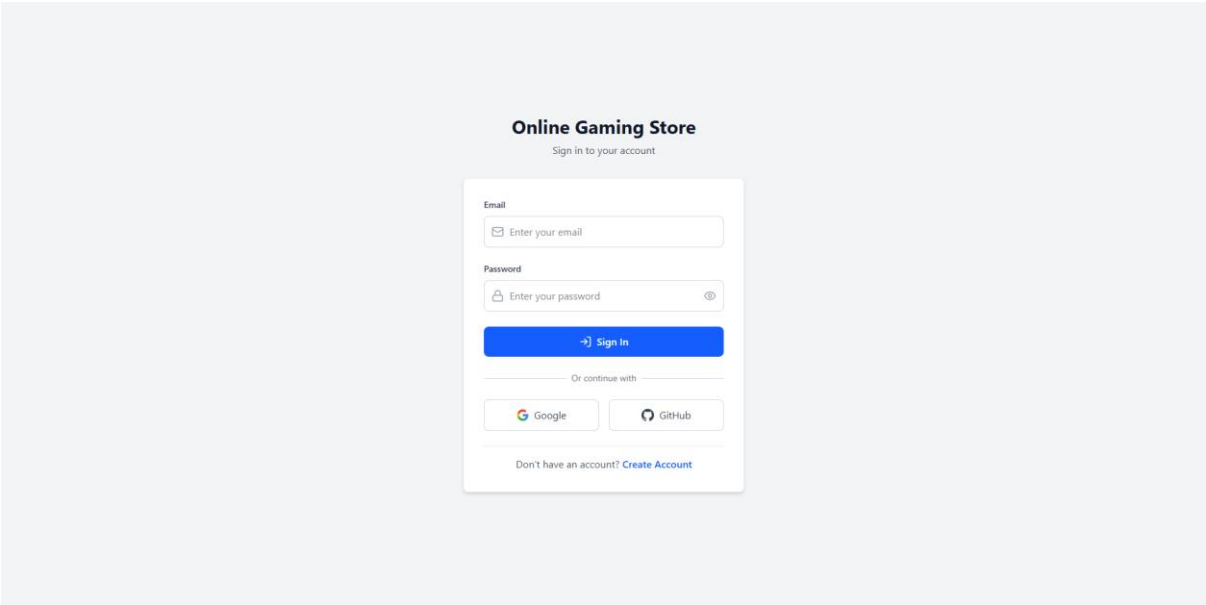
Sr. No	Field	Datatype	Constrain	Description
1	roleId	Number(4)	Primary Key	For Uniquely Identification
2	role	Varchar(10)	Not Null	Define Role

9. User : To store User details and Credential For Authenticate and Authorized

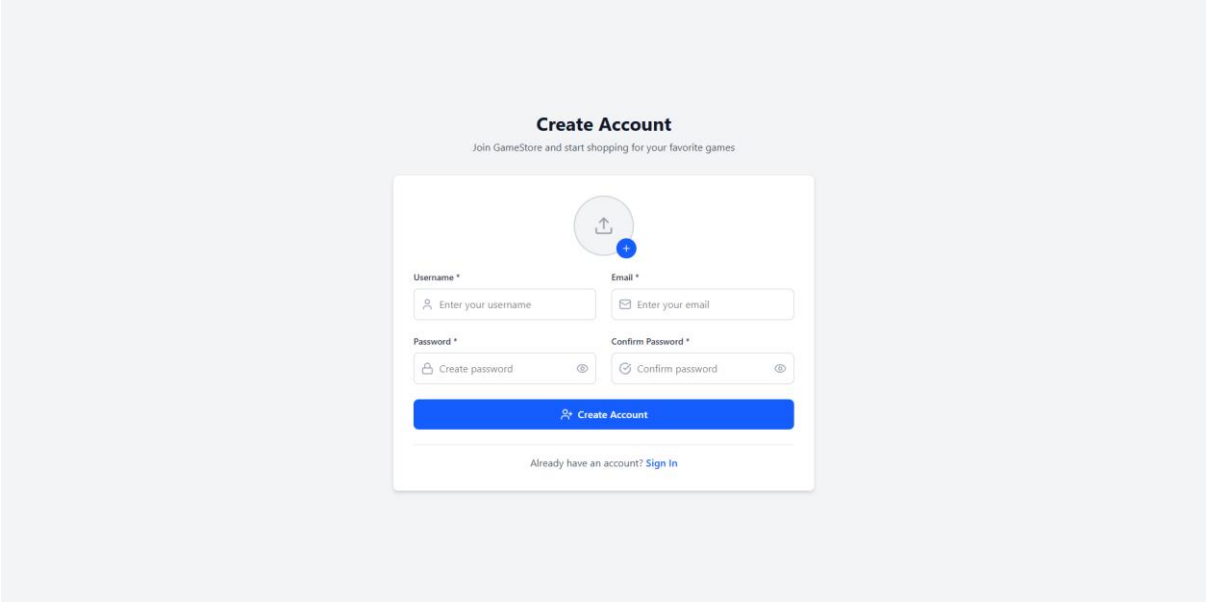
Sr. No	Field	Datatype	Constrain	Description
1	userId	Number(4)	Primary Key	For Uniquely Identification
2	userame	Varchar(20)	Unique	For Login Purpose
3	password	Varchar(500)	Not Null	TO Store Encrypted Password
4	roleId	Number(4)	Reference(Role)	To Store Role Id
5	fullname	Varchar(100)		TO Customer Full Name
6	Phoneno	Varchar(10)	Unique	TO Store User Phone Number
7	City	Varchar(50)	Not Null	To Store City
8	Address	Varchar(300)		To Store Full Address
9	Country	Varchar(50)		To Store Country

8. IMPLEMENTATION OF SYSTEM

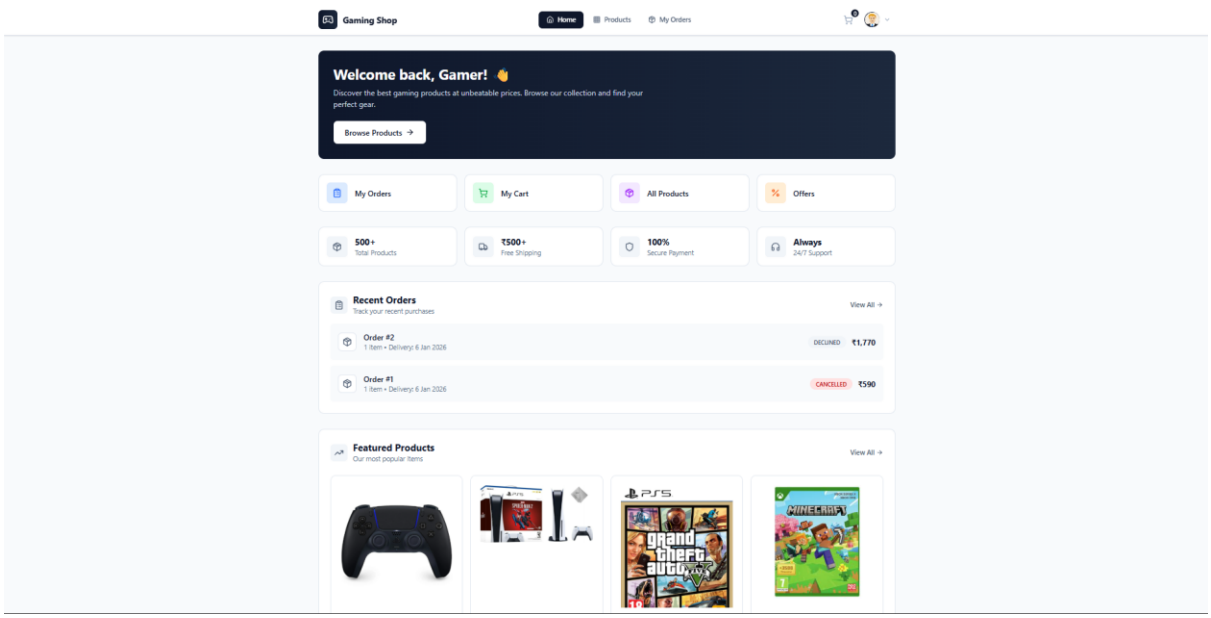
8.1 Screen Snapshots :



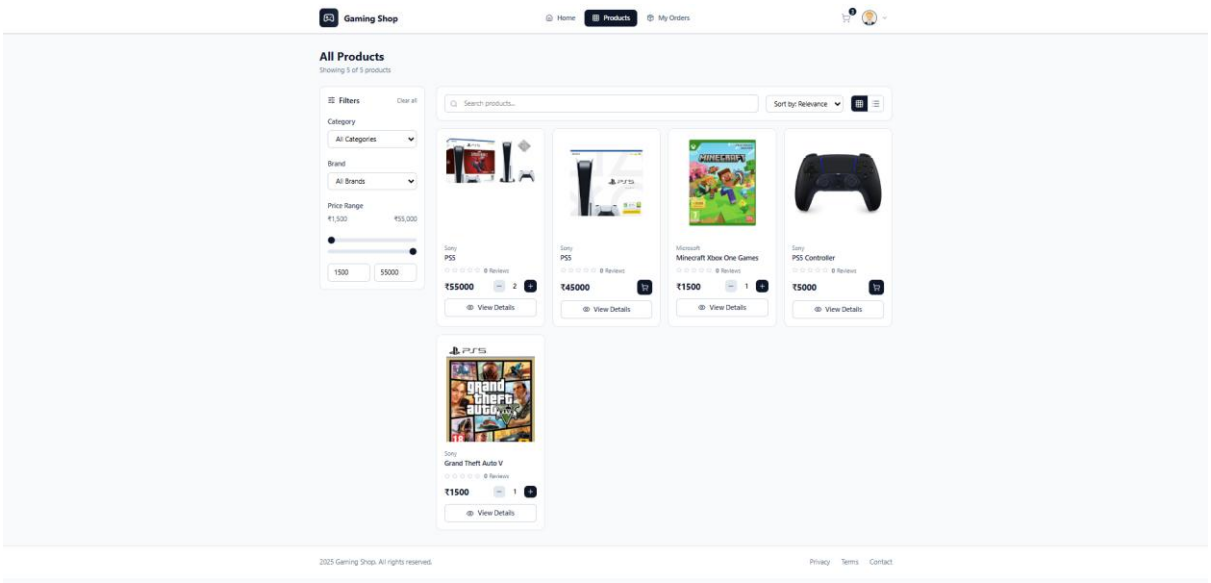
- This is Login screen to get enter



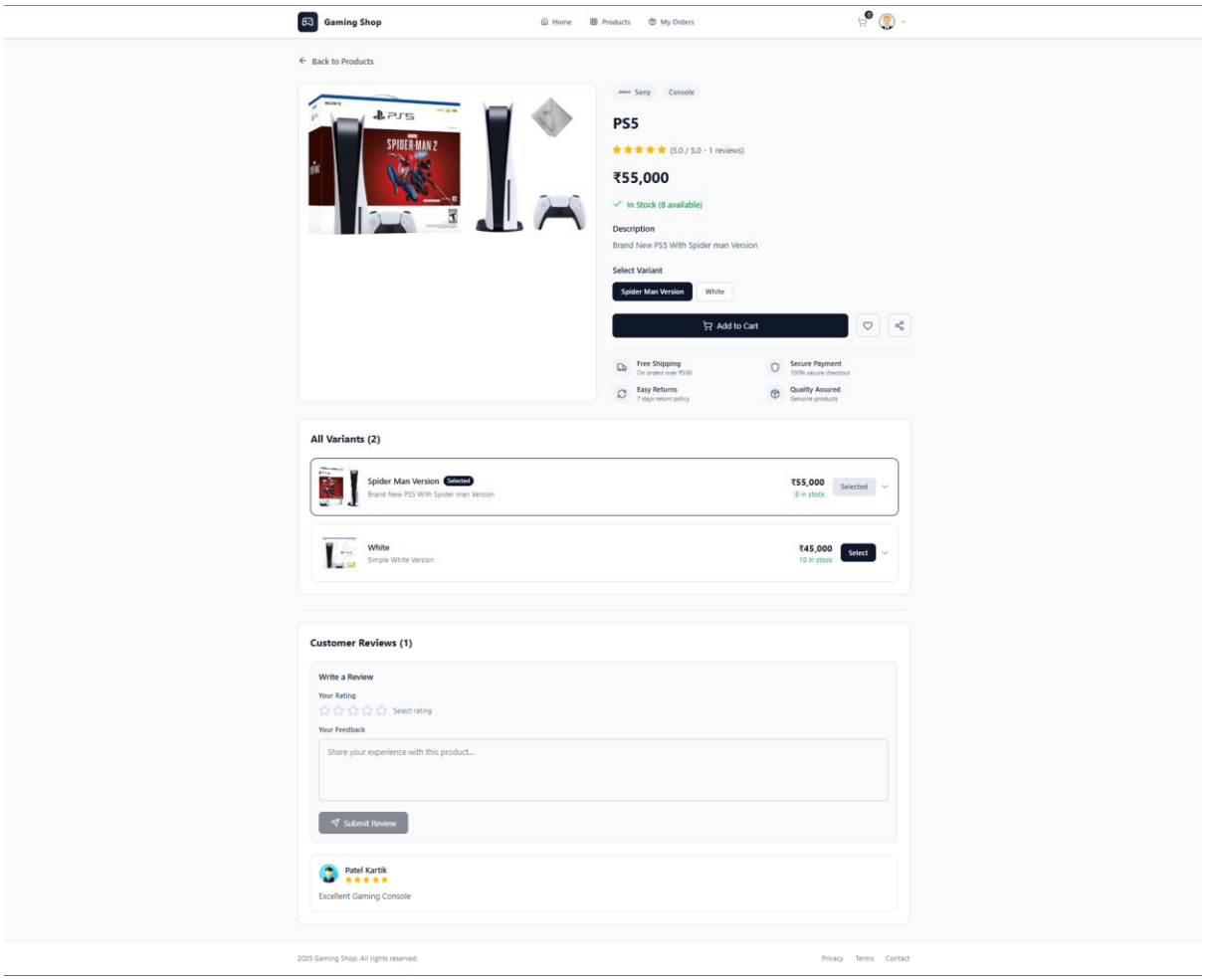
- This is Registration Page to create new Account for system



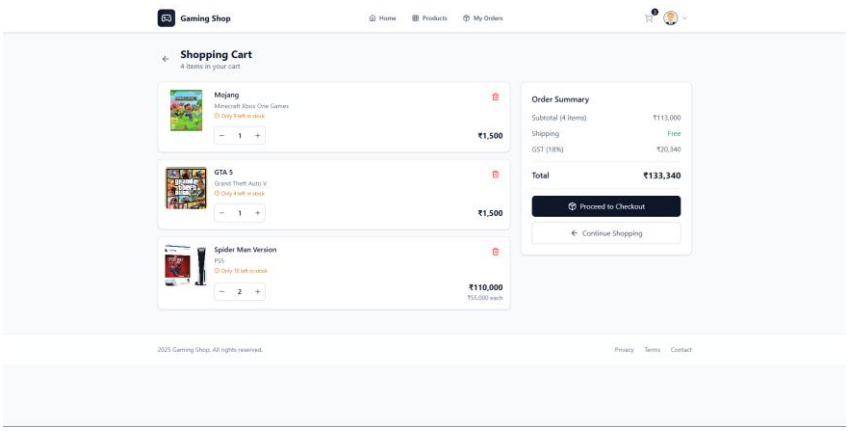
- This is Home Page of the System for customer



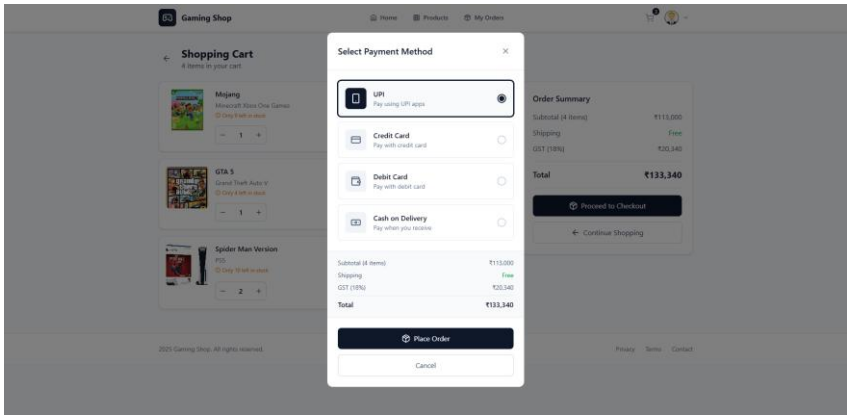
- This is Product listing Page of the System for customers



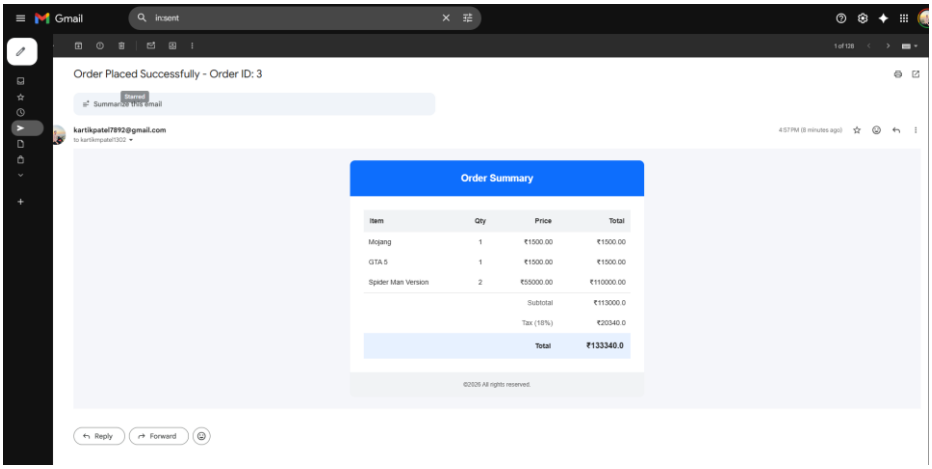
- Single Product Page with variants



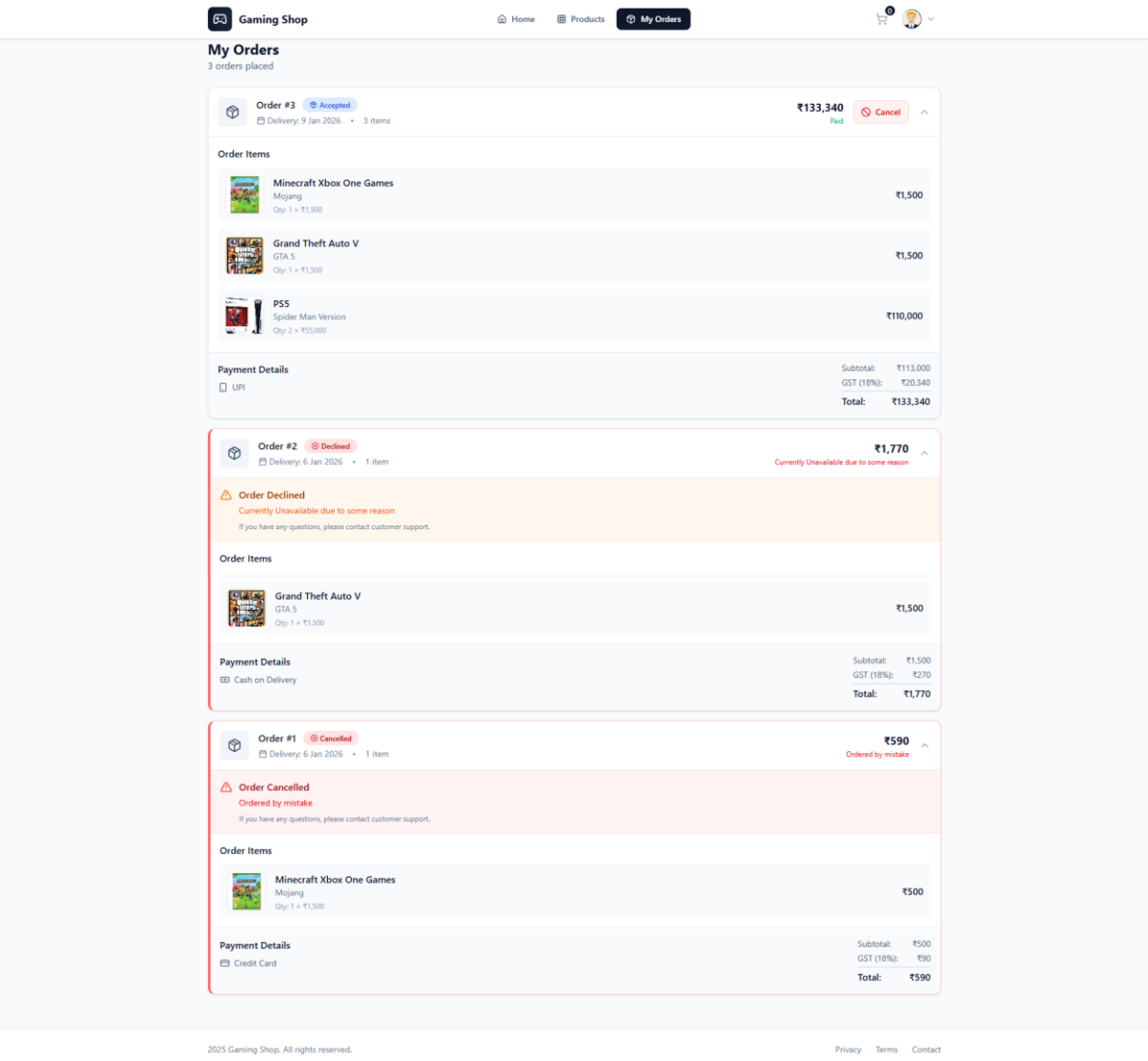
- Product in Cart Page for checkout The Order of selected items



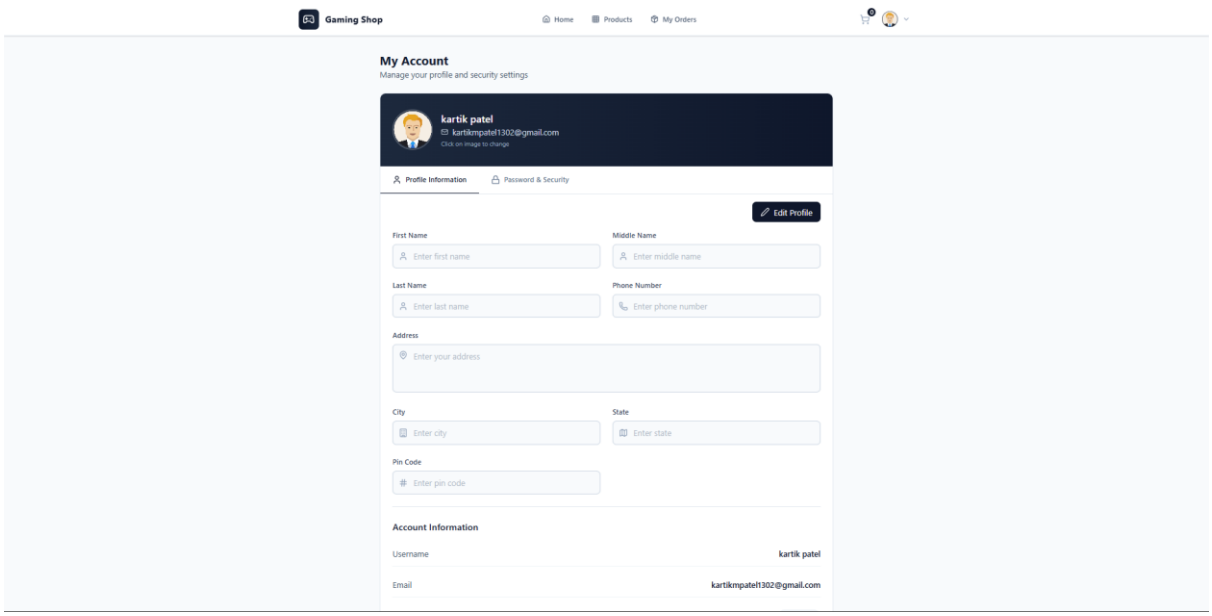
- Checkout with Case on Delivery Option.



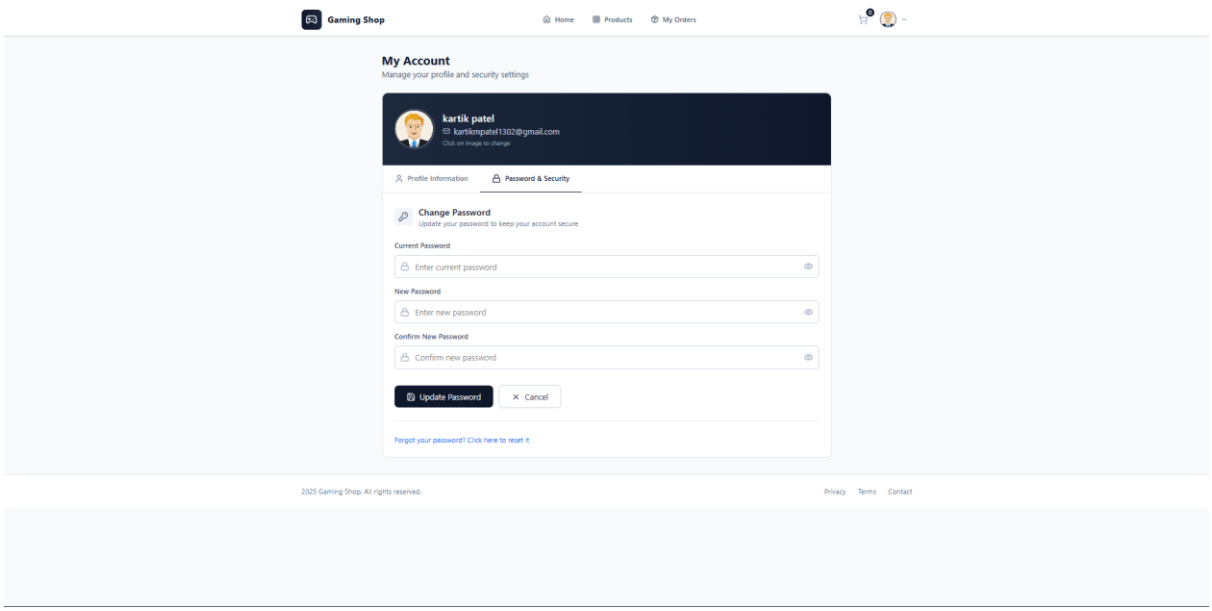
- Mail Will Be Send After Order Placed



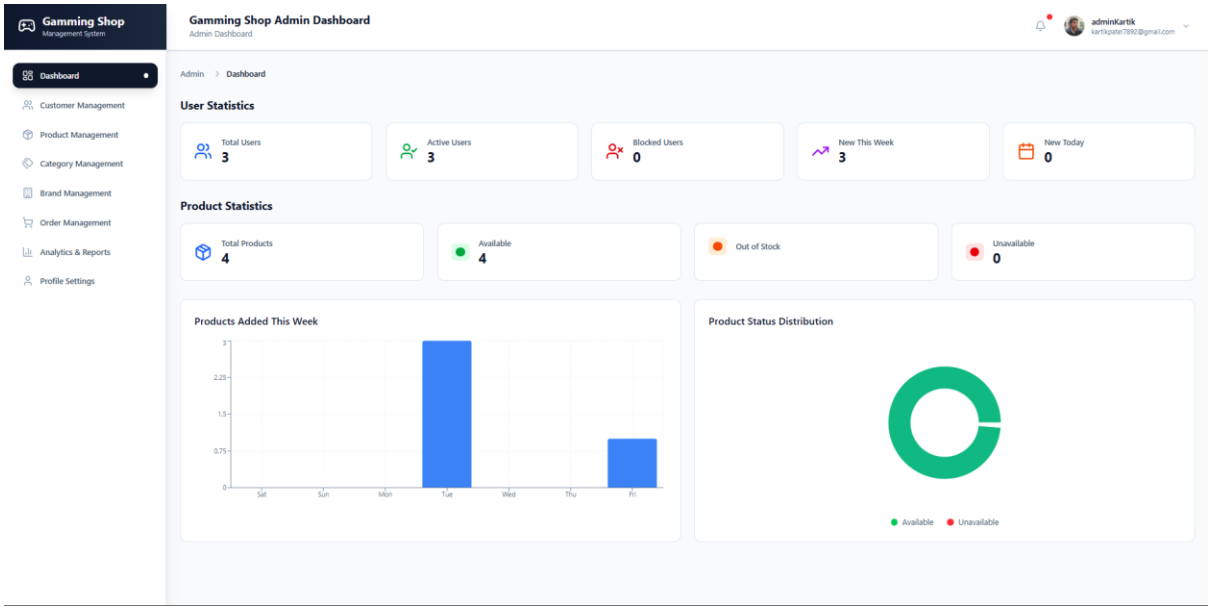
- Order History of Logged in user.



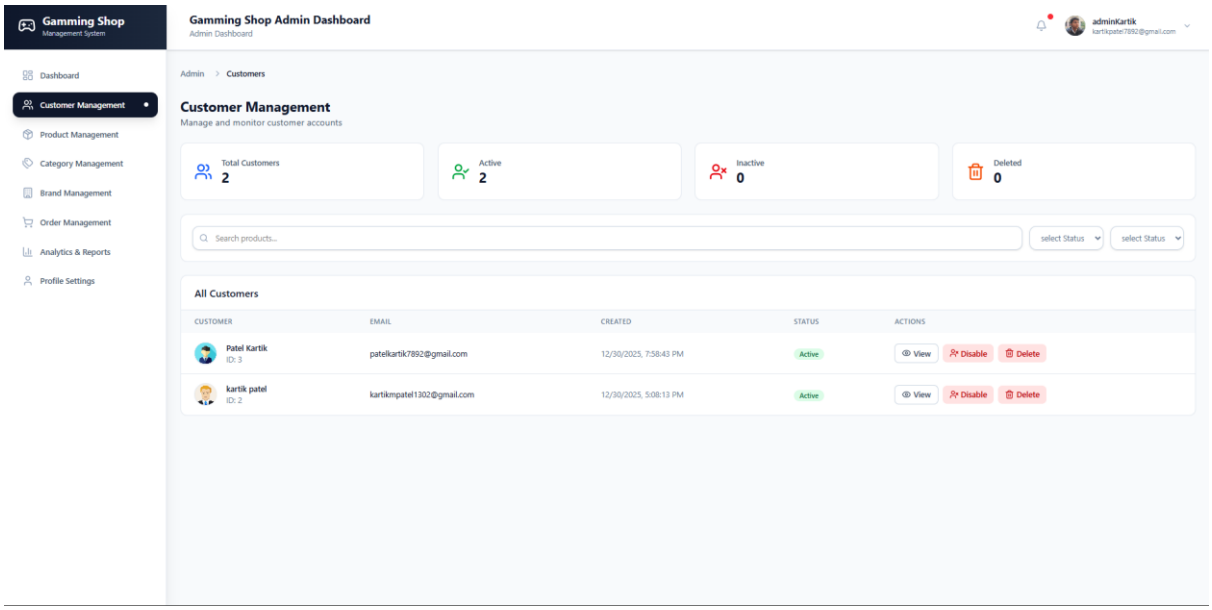
- Profile Of Current Logged in user for edit details



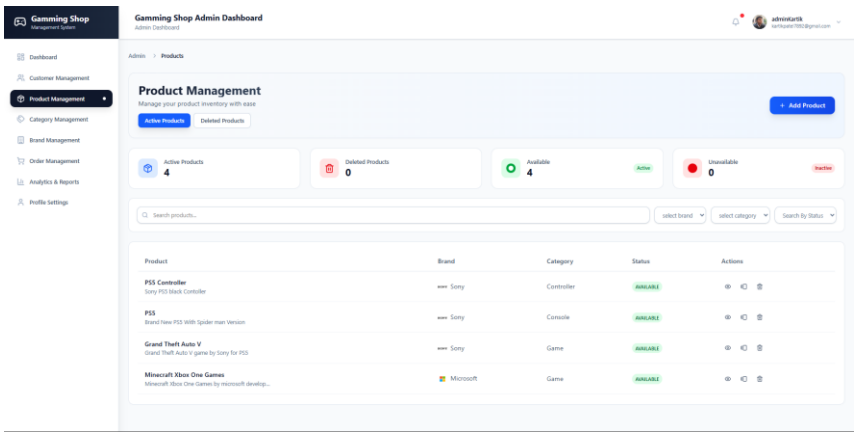
- Change Password of logged in user



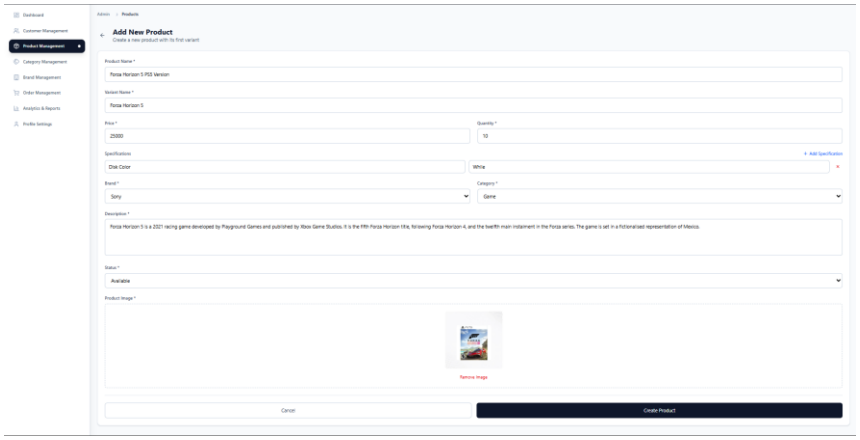
➢ Admin Dashboard with system data and activity



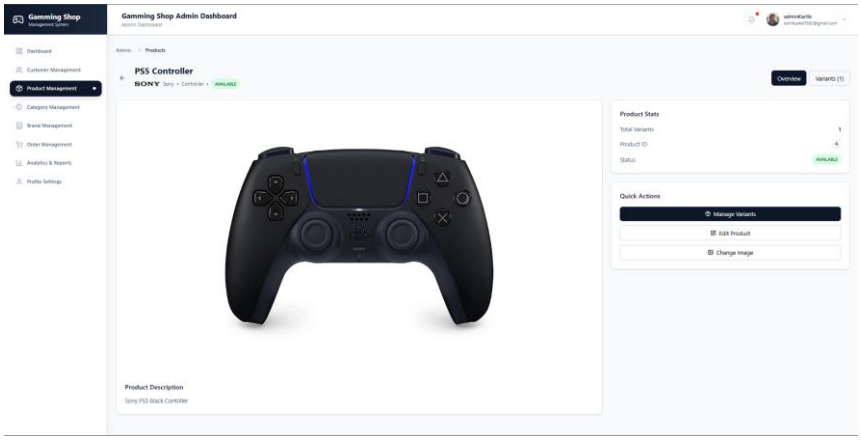
➢ User Management Page of Admin Panel



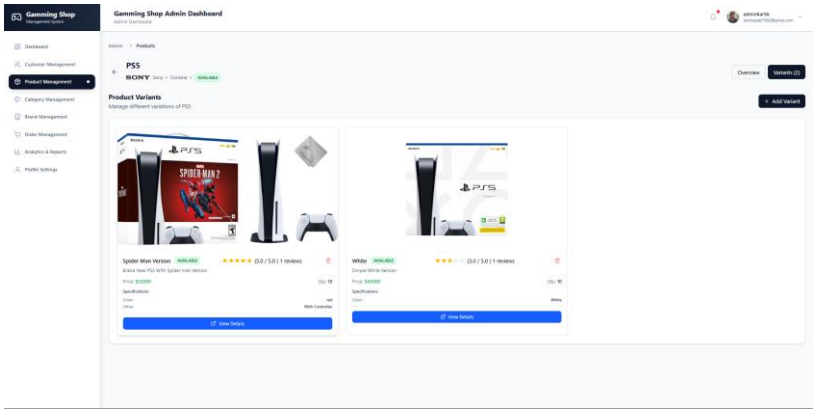
➢ Product Management Page of the Admin Panel



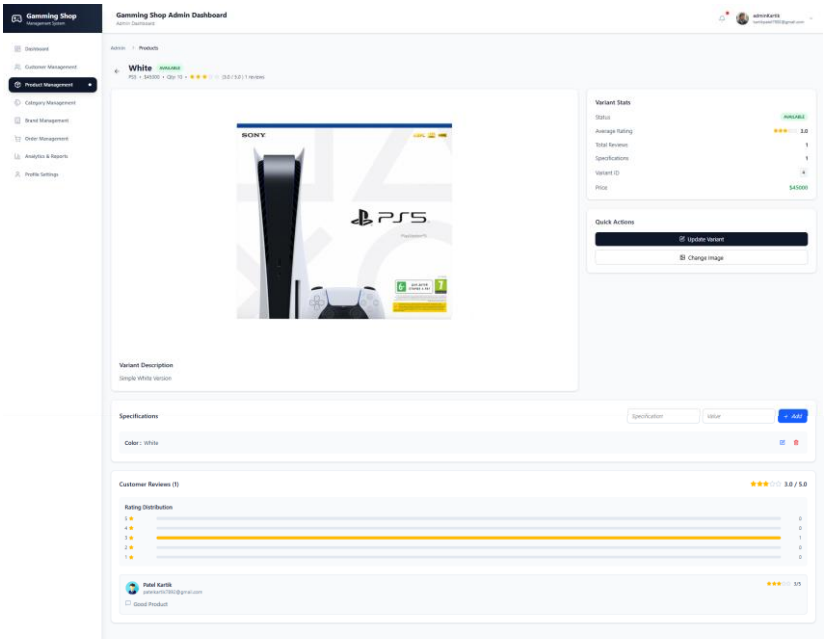
➢ Product Edit Page of the Admin Panel



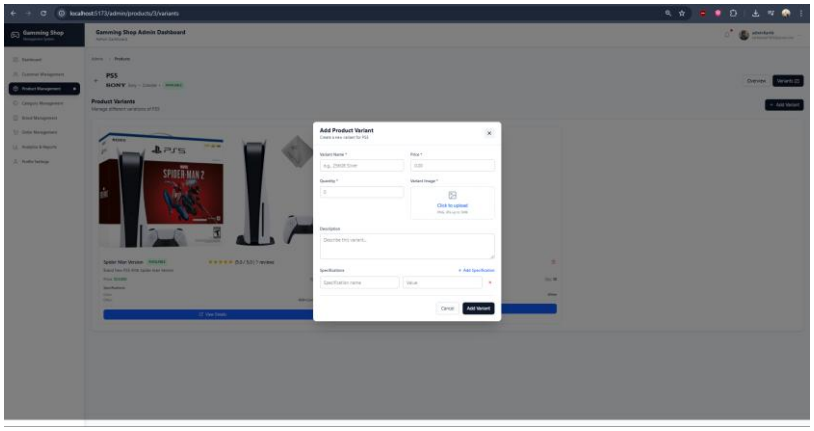
➢ Product Detail Page of the Admin Panel



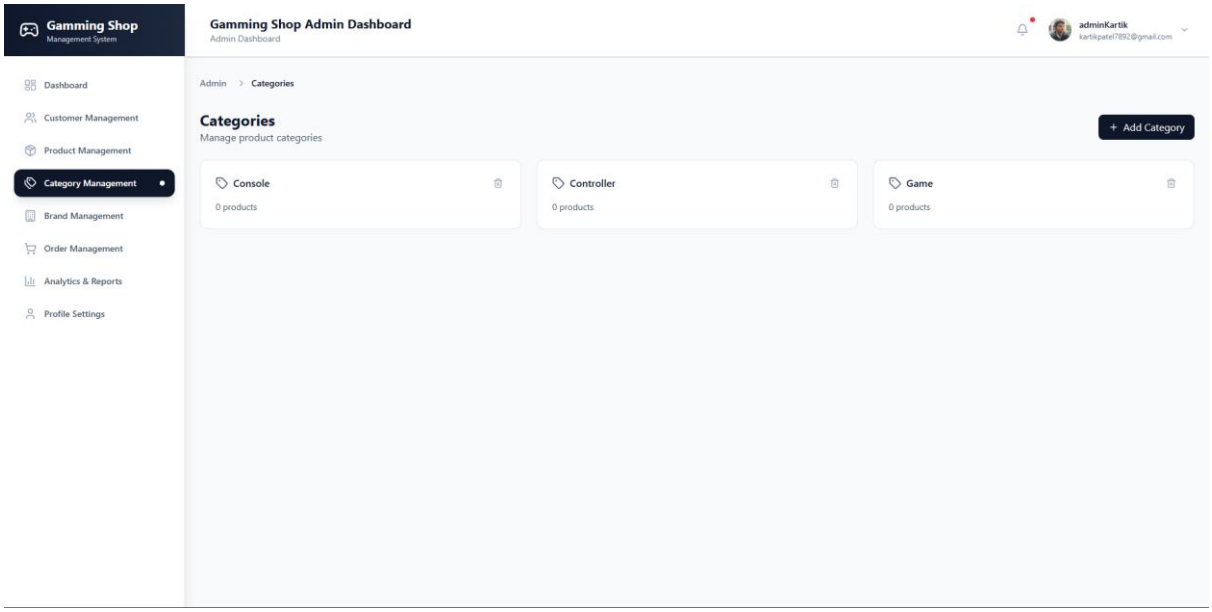
- Product variant Listing Page



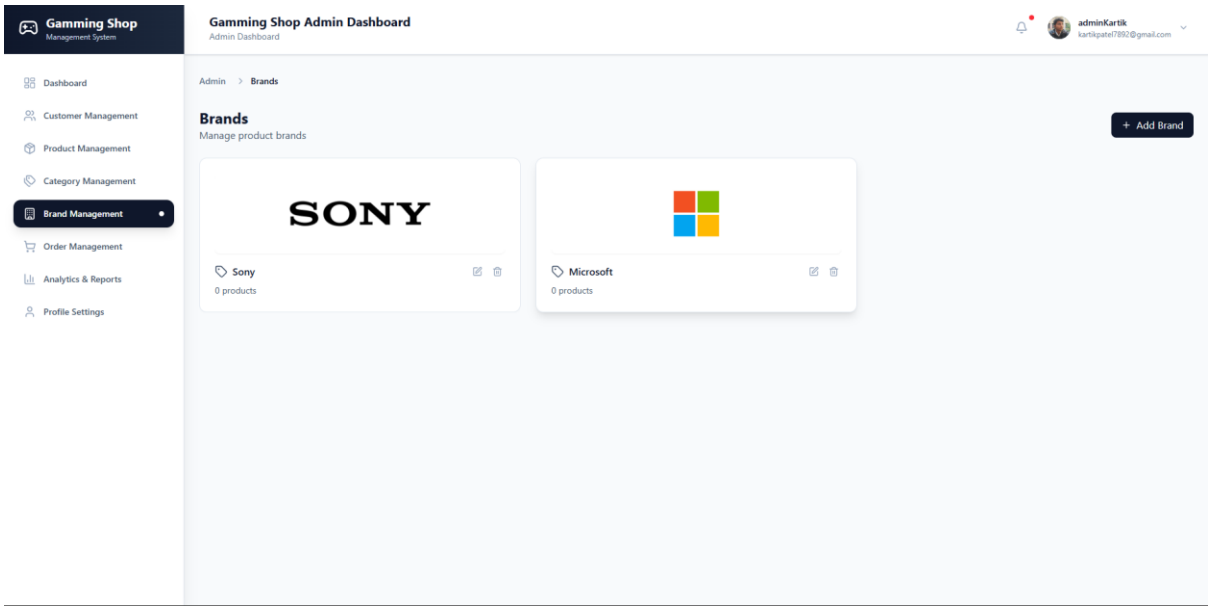
- Product variant Detail Page



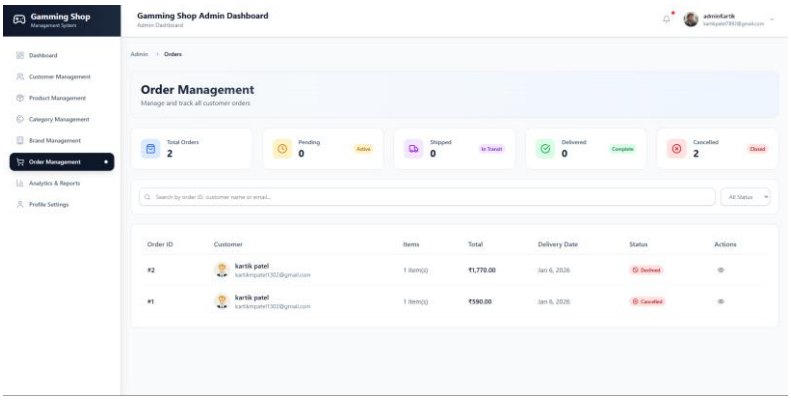
- Product variant Edit Popup



➢ Category Management Page Admin Panel



➢ Brand Management Page Admin Panel



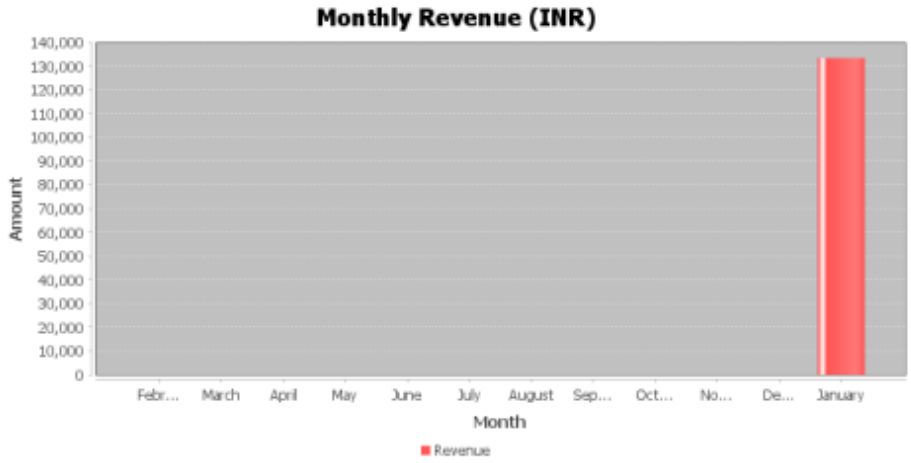
- Order Management Page Admin panel

8.2 Reports :

8.2.1 Sales Report :

Online Shopping Management System
Annual Sales Report
Currency: INR

Sr.No	Month	Revenue (INR)
1	February	0.0
2	March	0.0
3	April	0.0
4	May	0.0
5	June	0.0
6	July	0.0
7	August	0.0
8	September	0.0
9	October	0.0
10	November	0.0
11	December	0.0
12	January	133340.0



Generated by Online Shopping System

- Above Pdf will be generated by the system

8.2.2 Inventory Report :

Online Shopping Management System
Inventory Stock Report

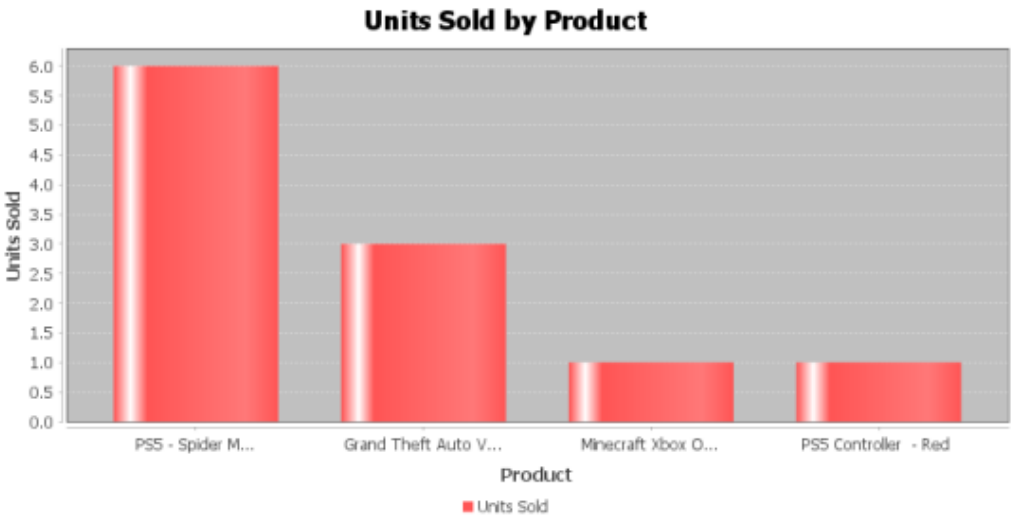
Sr.No	Product	Category	Variant	Stock	Status
1	Grand Theft Auto V	Game	GTA 5	0	OUTOFSTOCK
2	Minecraft Xbox One Games	Game	Mojang	15	AVAILABLE
3	PS5 Controller	Controller	Red	19	AVAILABLE
4	PS5	Console	Spider Man Version	30	UNAVAILABLE
5	PS5	Console	White	30	AVAILABLE
6	PS5 Controller	Controller	Black	50	AVAILABLE

- Above PDF will be generated by the System

8.2.3 Product Performance Report :

Online Shopping Management System
Product Performance Report
Currency: INR

Sr.No	Product	Units Sold	Revenue (INR)	Avg Rating
1	PS5 - Spider Man Version	6	330000.0	3.7
2	Grand Theft Auto V - GTA 5	3	4500.0	4.0
3	Minecraft Xbox One Games - Mojang	1	1500.0	4.0
4	PS5 Controller - Red	1	5000.0	4.0



- Above PDF will be generated by the System

8.3 Testing :

8.3.1 Testing Strategy :

- **Black-Box Testing**

Black box testing is a testing technique in which the internal workings of the software are not known to the tester. The tester only focuses on the input and output of the software. Black box testing is mainly focused on testing the functionality of the software, ensuring that it meets the requirements and specifications. Black box testing uses methods like equivalence partitioning, boundary value analysis, and error guessing to create test cases.

8.3.2 Test cases and results :

1. **User Registration – Field Level Test Cases :**

Test case ID	Form/Screen	Field Name	Test Scenario	Input Data	Expected Result	Actual Result	Status (Pass / Fail)
1	Registration Form	UserName	Valid Name	Customer1	Accepted	Accepted	Pass
2	Registration Form	UserName	Empty Name		Validation Error	Validation Error	Pass
3	Registration Form	UserName	Existing Name	Customer1	Server Side Error	Server Side Error	Pass
4	Registration Form	UserName	Special Characters	@@@@	Validation Error	Validation Error	Pass
5	Registration Form	Email	Valid Email	customer1@gmail.com	Accepted	Accepted	Pass
6	Registration Form	Email	Empty Email		Validation Error	Validation Error	Pass
7	Registration Form	Email	Invalid Email Format	customer@gmail	Validation Error	Validation Error	Pass
8	Registration Form	Email	Existing Email	customer1@gmail.com	Server Side Error	Server Side Error	Pass
9	Registration Form	Password	Valid Password	Customer.018	Accepted	Accepted	Pass
10	Registration Form	Password	Empty Password		Validation Error	Validation Error	Pass
11	Registration Form	Password	Weak Password	password	Validation Error	Validation Error	Pass
12	Registration Form	Image	Correct Image	img.jpg	Accepted	Accepted	Pass
13	Registration Form	Image	Incorrect Format	img.txt	Validation Error	Validation Error	Pass
14	Registration Form	Image	Empty Image		Validation Error	Validation Error	Pass

Test case ID	Form/Screen	Field Name	Test Scenario	Input Data	Expected Result	Actual Result	Status (Pass / Fail)
15	Registration Form	Confirm Password	Match Password	Customer.018	Accepted	Accepted	Pass
16	Registration Form	Confirm Password	Password Mismatch	Customer018	Validation Error	Validation Error	Pass

2. User Login – Field Level Test Cases :

Test case ID	Form/Screen	Field Name	Test Scenario	Input Data	Expected Result	Actual Result	Status (Pass / Fail)
1	Login Form	Email	Valid Email	admin@gmail.com	Accepted	Accepted	Pass
2	Login Form	Email	Empty Email		Validation Error	Validation Error	Pass
3	Login Form	Email	Invalid Email Format	admin@gmail	Validation Error	Validation Error	Pass
4	Login Form	Password	Correct Password	@dmin.018	Login Successfully	Login Successfully	Pass
5	Login Form	Password	Invalid Password	Admin018	Login Failed	Login Failed	Pass
6	Login Form	Password	Empty Password		Validation Error	Validation Error	Pass
7	Login Form	Login Button	Valid Credentials	admin@gmail.com / @dmin.018	Login Successfully	Login Successfully	Pass
8	Login Form	Login Button	Invalid Credentials	admin@gmail.com / wrongpass	Login Failed	Login Failed	Pass

3. Admin – Add Product – Field Level Test Cases :

Test case ID	Form/Screen	Field Name	Test Scenario	Input Data	Expected Result	Actual Result	Status (Pass / Fail)
1	Add Product Form	Product Name	Valid Name	PS5	Accepted	Accepted	Pass
2	Add Product Form	Product Name	Empty Name		Validation Error	Validation Error	Pass
3	Add Product Form	Variant Name	Valid Name	Spider-Man Edition	Accepted	Accepted	Pass
4	Add Product Form	Price	Valid Price	50000	Accepted	Accepted	Pass
5	Add Product Form	Price	Negative Price	-500	Validation Error	Validation Error	Pass
6	Add Product Form	Quantity	Valid Quantity	10	Accepted	Accepted	Pass
7	Add Product Form	Quantity	Zero Quantity	0	Validation Error	Validation Error	Pass
8	Add Product Form	Specification	RAM	8GB	Accepted	Accepted	Pass
9	Add Product Form	Brand	Select Valid Brand	Sony	Accepted	Accepted	Pass
10	Add Product Form	Category	Select Valid Category	Console	Accepted	Accepted	Pass
11	Add Product Form	Description	Valid Description	High performance console	Accepted	Accepted	Pass
12	Add Product Form	Status	Valid Status	Available	Accepted	Accepted	Pass
13	Add Product Form	Variant Image	Valid Image	variant1.png	Accepted	Accepted	Pass
14	Add Product Form	Variant Image	Invalid Format	variant1.txt	Validation Error	Validation Error	Pass

4. **Admin – Add Variant – Field Level Test Cases :**

Test case ID	Form/Screen	Field Name	Test Scenario	Input Data	Expected Result	Actual Result	Status (Pass / Fail)
1	Add Variant Form	Variant Name	Valid Variant Name	Digital Edition	Accepted	Accepted	Pass
2	Add Variant Form	Variant Name	Empty Variant Name		Validation Error	Validation Error	Pass
3	Add Variant Form	Variant Price	Valid Price	45000	Accepted	Accepted	Pass
4	Add Variant Form	Variant Price	Invalid Price	-100	Validation Error	Validation Error	Pass
5	Add Variant Form	Variant Image	Valid Image	variant2.png	Accepted	Accepted	Pass
6	Add Variant Form	Variant Image	Invalid Format	variant2.txt	Validation Error	Validation Error	Pass

5. **Checkout – Field Level Test Cases**

Test case ID	Form/Screen	Field Name	Test Scenario	Input Data	Expected Result	Actual Result	Status (Pass / Fail)
1	Checkout Page	Address	Valid Address	Bhavnagar, Gujarat	Accepted	Accepted	Pass
2	Checkout Page	Address	Empty Address		Validation Error	Validation Error	Pass
3	Checkout Page	Payment Method	Select Payment Method	Case on Delivery	Accepted	Accepted	Pass
4	Checkout Page	Payment Method	No Payment Selected		Validation Error	Validation Error	Pass
5	Checkout Page	Place Order	Valid Order	Order Data	Order Placed	Order Placed	Pass
6	Checkout Page	Payment	Invalid Payment	Declined Card	Payment Failed	Payment Failed	Pass
7	Checkout Page	Payment	Successful Payment	Select Case on Delivery	Order Confirmed	Order Confirmed	Pass

9. FUTURE ENHANCEMENT

- Mobile Application Support
- Multi-Level Approval Workflow
- AI-Based Resume Screening
- Real-Time Notifications System

10. BIBLIOGRAPHY

- Spring Framework. (2024).
  Spring Boot documentation. Spring.io.https://spring.io/projects/spring-boot

- Oracle Corporation. (2024). Java SE documentation.
  Oracle Documentation. https://docs.oracle.com/en/java/javase/

- Oracle Corporation. (2024). MySQL documentation.
  MySQL Official Documentation. https://dev.mysql.com/doc/

- Postman Inc. (2024). Postman API Platform.
  Postman Learning Center.https://learning.postman.com/docs/